



UNIVERSITÀ DEGLI STUDI DI MILANO
FACOLTÀ DI SCIENZE E TECNOLOGIE

Corso di Laurea in Sicurezza dei Sistemi e delle Reti Informatiche

Refactoring e implementazione di
servizi di Machine Learning per
l'analisi dati in un contesto distribuito
basato su Kubernetes

Relatore

Prof. Claudio A. Ardagna

Correlatori

Prof. Marco Anisetti

Dott. Antongiaco Polimeno

Tesi di laurea di

Gabriele Stentella

984491

Anno Accademico 2023/2024

Prefazione

A partire dai primi anni 2000 si sono sviluppate numerose tecnologie per l'elaborazione distribuita di *Big Data*. Il grande numero di tecnologie differenti ha fatto sì che venissero sviluppati software strettamente legati all'infrastruttura di esecuzione.

Fra il 2003 e il 2008, Google divulgò tre componenti fondamentali sviluppati per affrontare la sfida di indicizzare internet: la tecnologia del proprio file system [17], il modello MapReduce [12] e il sistema di archiviazione dati distribuito Bigtable [7]. Tali tecnologie gettarono le basi per lo sviluppo del framework *Hadoop*, uno strumento *open source* che ha consentito una larga adozione di tecniche per la computazione distribuita. Negli anni successivi, si diffuse l'utilizzo di *Hadoop* e si sviluppò un ecosistema con tale tecnologia al centro, con file system distribuiti e framework per convertire le classiche interrogazioni *SQL* in lavori MapReduce. Dal momento che avviare e gestire un cluster *Hadoop* era un'operazione costosa e impegnativa, la vera adozione di massa avvenne quando Amazon lanciò il suo servizio di cloud computing *on-demand* e contestualmente *Elastic MapReduce (EMR)*, consentendo di sfruttare i vantaggi del modello MapReduce impiegando il file system distribuito *Amazon Simple Storage Service (S3)* senza dover acquistare e configurare l'hardware necessario.

Alcuni anni dopo vennero rilasciate le prime versioni di *Apache Spark*, un nuovo framework con una sintassi più semplice e migliori performance di MapReduce, grazie alla sua capacità di utilizzare la RAM come cache. La gestione delle risorse del cluster era gestita attraverso il resource manager *Hadoop Yarn*. Poco dopo, Spark cominciò ad avere successo anche nel mondo del Machine Learning (ML), grazie al framework *Apache Mahout* prima, e poi alla *Machine Learning library (MLlib)*, più veloce. Le possibilità offerte da Spark si ampliarono grazie alla nascita di scheduler come *Apache Airflow* e *Luigi*.

Fra il 2013 e il 2014 si diffusero *Docker* e l'orchestratore *Kubernetes* che semplificarono la gestione di cluster anche di grandi dimensioni, rendendo le architetture molto più stabili e scalabili. *Spark* si è adattato al contesto, permettendo di utilizzare proprio *Kubernetes* come resource manager.

Con l'avvento del Deep Learning (DL) le attività di computazione si sono spostate sull'utilizzo massiccio di GPU e sull'impiego delle più importanti librerie di *Python* dedicate, *Tensorflow* e *Pytorch*. Per questi carichi di lavoro si è passati all'utilizzo delle Application Programming Interface (API) proprietarie delle librerie per distribuire la computazione anziché usare Spark, in modo da sfruttare in maniera efficiente l'hardware.

Questa tesi si propone di esaminare come eseguire algoritmi che utilizzano tecnologie diverse —ossia Spark e Tensorflow— distribuendo la computazione su un'unica infrastruttura. Con l'approccio illustrato nell'elaborato, il codice degli algoritmi è disaccoppiato dal cluster di esecuzione, dunque si possono riutilizzare i modelli su diverse infrastrutture.

Il percorso che si intende illustrare in questo elaborato ha previsto sia una fase di recupero di algoritmi ML pre-esistenti per la loro esecuzione in un ambiente distribuito attraverso Kubernetes, sia l'implementazione di nuovi algoritmi che sfruttano le API delle moderne librerie per lo sviluppo di algoritmi di DL. Nell'introduzione (1) viene analizzato il contesto in cui si inserisce il lavoro di tesi, in modo tale da illustrare nel dettaglio le motivazioni alla base dell'attività svolta, per poi descrivere il contributo dato dalla soluzione proposta e dettagliare la struttura della trattazione che segue. Nel capitolo 2 si introducono gli aspetti tecnici alla base del lavoro svolto. Il capitolo 3 fornisce una panoramica del flusso di lavoro della tesi. Nel capitolo 4 si discutono lo scenario di partenza e gli algoritmi per l'analisi dati. Nei capitoli 5, 6, 7 si descrive in dettaglio il lavoro svolto durante il tirocinio e si analizza il contributo di questa tesi, inoltre si riporta la fase di testing. Con il capitolo 8 si conclude l'elaborato, esplorando gli sviluppi futuri.

Indice

1	Introduzione	1
2	Le tecnologie per l'analisi dati e il calcolo distribuito	4
1	L'IA per l'analisi dati	4
1.1	Tensorflow	5
2	Apache Spark	6
3	Docker	10
4	Kubernetes	11
3	Descrizione del lavoro svolto	14
4	Scenario di partenza	17
1	Analisi e operazioni preliminari	22
5	Esecuzione in ambiente gestito con Kubernetes	26
1	Esecuzione su cluster locale	26
2	Esecuzione su cluster AWS EKS	28
6	Studio e implementazione di un algoritmo di test in Tensorflow	33
1	Analisi	33
2	Configurazione dell'ambiente di lavoro	34
3	Implementazione	35
3.1	Deployment su cluster Kubernetes	36
7	Deployment sull'infrastruttura MUSA e testing	40
1	Architettura MUSA	40
2	Processo di deployment	41
3	Testing e valutazione delle performance in ambiente distribuito	42
3.1	Spark task	43
3.2	Tensorflow task	44
8	Conclusioni	46

A	POM generale	48
B	POM di un singolo modulo	49
C	Identity and Access Management (IAM) policies	52
D	Configurazione di EKS StartJobRun	55
E	Estratto dai log di un'esecuzione su cluster EKS	57
	Bibliografia	64

Elenco delle figure

3.1	Flusso di lavoro	16
5.1	Funzionamento di spark-submit	27
5.2	Schema riassuntivo dell'infrastruttura configurata con Amazon EKS	32
6.1	Schema di funzionamento della distribuzione di tensorflow su cluster Kubernetes	38
7.1	Meccanismo per l'implementazione dell'autenticazione basata sui ruoli	41

Capitolo 1

Introduzione

La presenza sempre più diffusa dei *Big Data* ha reso necessario avere infrastrutture e tecnologie di analisi adatte ad estrarre valore da tali dati e che siano scalabili per essere utilizzate da realtà di diverse dimensioni. Al fine di avere procedure efficaci sia nell'attività di analisi che in quella di costruzione di modelli predittivi, occorre affrontare i problemi tipici dei *Big Data* che sono la diffusa eterogeneità, l'accumulo di rumore in fase di raccolta, la frequente presenza di correlazioni spurie. Gli algoritmi impiegati devono essere anche accurati dal punto di vista statistico ed efficaci per quanto riguarda il carico computazionale. Per tutte queste ragioni, ma in modo particolare per la necessità di efficienza computazionale, i *Big Data* hanno portato allo sviluppo di nuove infrastrutture di computazione e nuovi metodi di memorizzazione dei dati [16].

In tale contesto, le tecnologie di *Apache Hadoop* prima e *Apache Spark* poi, hanno reso semplice ed economica l'analisi e la memorizzazione dei dataset di grandi dimensioni. Dal momento che l'analisi di *Big Data* richiede una sperimentazione reiterata per trovare i parametri corretti e individuare le giuste metriche di valutazione, Spark rappresenta una soluzione tecnologicamente adatta a supportare in modo efficiente questo tipo di lavoro, in particolare se viene utilizzata la libreria MLlib per il *design* e il *tuning* di algoritmi ML [28].

Negli ultimi anni, oltre agli algoritmi ML di *shallow learning*, hanno avuto molto successo i modelli DL che, a differenza delle architetture *shallow*, traggono vantaggio dall'avere un grande numero di parametri liberi, quindi l'addestramento di tali modelli vede nei *Big Data* una risorsa fondamentale, e contemporaneamente necessita di metodi efficienti per la parallelizzazione delle operazioni di training [37].

I benefici dati da un'analisi svolta con l'ausilio di algoritmi ML o modelli DL sono evidenti nel campo della ricerca medica e scientifica, nella gestione di infrastrutture critiche e nella cosiddetta *business intelligence* [20], ma la lette-

ratura è sempre più ricca di applicazioni di queste tecnologie nel campo della sicurezza informatica. Grazie alla presenza pervasiva dell'Internet of Things (IoT) e in generale dell'evoluzione tecnologica dei dispositivi informatici, compresi gli strumenti di rete, c'è una grandissima produzione di dati che possono essere analizzati in tempo reale per eseguire azioni di prevenzione, ricerca di minacce e *incident response*. Soltanto alcune fra le numerose applicazioni in questo campo sono l'analisi del traffico di rete con l'obiettivo di istituire uno schema di priorità e rendere meno efficaci eventuali attacchi Denial of Service (DoS), identificare comportamenti sospetti, individuare malware e integrare i sistemi di autenticazione [29]. In particolare, l'uso dei classificatori è utile per implementare dei filtri sia sul traffico web sia sulla corrispondenza di posta elettronica, in modo tale da ridurre la probabilità che il personale di un'azienda entri in contatto con contenuti malevoli, e di conseguenza minimizzare l'impatto di email spam o siti web malevoli sulla rete aziendale o di un'infrastruttura critica. I classificatori in questione devono essere allenati con un insieme di dati molto grande e in continuo aggiornamento, in modo tale che stiano al passo con l'evoluzione delle minacce. Inoltre è necessario che, in base al tipo di classificazione, venga utilizzato un modello statistico adeguato.

L'obiettivo del lavoro di tesi è quello di recuperare degli algoritmi ML per l'analisi dati —sviluppati in linguaggio Java e utilizzando la tecnologia Spark— con il fine di renderli eseguibili in un ambiente distribuito, superando il resource manager originario Hadoop Yarn e utilizzando Kubernetes. Parallelamente al recupero degli algoritmi esistenti, la tesi si pone l'obiettivo di studiare la tecnologia di esecuzione distribuita della libreria *Tensorflow* per lo sviluppo di nuovi algoritmi per l'analisi dati eseguibili nel medesimo ambiente distribuito dei precedenti. L'infrastruttura finale —pensata per lo storage sicuro e l'elaborazione di Big Data— in cui si sono eseguiti gli algoritmi, è un ambiente distribuito amministrato con l'orchestratore Kubernetes ed è stata implementata nell'ambito del progetto Multilayered Urban Sustainability Action (MUSA) finanziato dal Ministero dell'Università e della Ricerca attraverso il Piano Nazionale di Ripresa e Resilienza.

Il progetto MUSA è focalizzato sulla creazione di una piattaforma per gestire flussi di lavoro intensivi per l'elaborazione di Big Data in un continuum Edge-Cloud. Grazie alla collaborazione con Internet Service Providers e aziende di telecomunicazioni, il progetto accede ad ampie risorse di rete e computazionali, e ciò consente pieno controllo dell'infrastruttura. Sfruttando l'infrastruttura di rete privata degli ISP e dei fornitori di telecomunicazioni, MUSA ottiene un equilibrio tra soluzioni basate su cloud e on-premises, offrendo vantaggi come l'alta disponibilità di risorse, l'accesso a una vasta gamma di risorse hardware e una maggiore privacy e sicurezza per i dati. La piattaforma è composta da tre elementi principali: un Cloud pubblico, una piattaforma di edge computing

abilitata dal 5G e una piattaforma di edge computing on-premises [4]. All'interno del progetto, il Cloud è di particolare interesse per questa tesi: si tratta di una piattaforma per il data modeling intensivo, sviluppato con pacchetti open source.

Il principale contributo di questo elaborato è lo studio della possibilità di distribuire la computazione di algoritmi già implementati, con modifiche minime o del tutto assenti alla logica dell'applicazione. Nel momento in cui il codice e l'infrastruttura di esecuzione sono disaccoppiati, infatti, il riuso di programmi già esistenti diventa molto più agevole e consente di utilizzare algoritmi come quelli coinvolti in questa tesi come componenti per sistemi più complessi. Per aggiungere un ulteriore elemento di disaccoppiamento fra codice ed infrastruttura, il lavoro si è concentrato anche sulla possibilità di eseguire in un unico ambiente —nel caso di specie gestito con Kubernetes— applicazioni che sfruttano tecnologie e linguaggi differenti.

L'elaborato è organizzato come segue: nel capitolo 2 si espone lo stato dell'arte delle tecnologie per l'analisi dati e il calcolo distribuito, in modo tale da introdurre gli aspetti tecnici alla base del lavoro; il capitolo 3 descrive il lavoro svolto; nel capitolo 4 si riporta la fase di analisi, dunque si illustrano gli algoritmi coinvolti nella tesi e le operazioni preliminari; nel capitolo 5 viene descritto nel dettaglio il deployment degli algoritmi pre-esistenti su cluster Kubernetes; il capitolo 6 analizza nel dettaglio la fase dedicata allo studio dell'implementazione di nuovi algoritmi con Tensorflow; con il capitolo 7 viene affrontato il deployment sull'architettura sviluppata all'interno del progetto MUSA, inoltre è riportato il processo di testing e di relativa valutazione delle performance degli algoritmi in contesto distribuito; nel capitolo 8 si riassume il contributo dato da questa tesi e si esplorano i possibili sviluppi futuri.

Capitolo 2

Le tecnologie per l'analisi dati e il calcolo distribuito

1 L'IA per l'analisi dati

Con le tecniche di analisi statistica convenzionali si ricerca un modello che descriva in modo appropriato i dati che si hanno a disposizione o, più frequentemente, le variabili del dataset ritenute significative. Attraverso approcci di Intelligenza Artificiale (IA), si utilizzano algoritmi e modelli computazionali per estrarre conoscenze, identificare pattern e prendere decisioni basate sui dati. Questi sistemi IA possono automatizzare processi analitici complessi, in particolare con grandi moli di dati. Infatti, dati di alta qualità e diversificati forniscono una base più solida per l'analisi, consentendo ai modelli di apprendimento automatico di identificare pattern più significativi e di generare risultati più precisi.

Gli algoritmi sono suddivisibili in due categorie sulla base del funzionamento in fase di apprendimento:

1. nel *supervised learning* i dati sono etichettati, quindi si hanno n esempi, ciascuno composto dal dato (x) e dalla relativa etichetta (y). L'obiettivo di un algoritmo di apprendimento in questa categoria è quello di trovare una funzione che associ ad ogni x la relativa y . Spesso la funzione in questione non serve per trovare con certezza matematica una certa y , ma restituisce il valore che ha probabilità maggiore di essere l'etichetta per un dato x ;
2. per lavorare con dati senza etichette associate si utilizzano algoritmi di *unsupervised learning* che cercano di costruire una rete con il comportamento desiderato in maniera iterativa, adattando i pesi interni al modello per avvicinarsi di volta in volta all'obiettivo.

Come appare evidente dal funzionamento delle diverse modalità di apprendimento, l'obiettivo dei principali algoritmi di *supervised learning* è quello di fare predizioni osservando nuovi dati, sulla base di modelli costruiti su una collezione di dati dedicati, mentre quello dei principali algoritmi di *unsupervised learning* è ottenere informazione utile estraendo le caratteristiche più interessanti (pattern, anomalie o trend particolari) da dataset di grandi dimensioni. Gli algoritmi predittivi possono essere utilizzati sia per assegnare i dati a specifiche categorie predeterminate (classification), ovvero fare previsioni con un output discreto, sia per identificare le relazioni presenti fra le diverse variabili (regression), ovvero effettuare previsioni con output continuo. L'individuazione dei pattern comuni all'interno di grandi collezioni di dati, invece, può essere sfruttata per molteplici obiettivi diversi: oltre alla costruzione di modelli si possono svolgere attività di preparazione dei dati oppure utili per analisi descrittive, per esempio, lavorando con *Big Data* può essere utile dividere il dataset in gruppi di oggetti con caratteristiche comuni (*cluster analysis*). Inoltre, dal momento che gli insiemi di dati hanno frequentemente un'elevata dimensionalità, sono spesso necessarie azioni che mirano alla riduzione dello spazio delle caratteristiche dei dati in questione, pur mantenendo le informazioni significative.

A tal proposito, è bene evidenziare che fra gli algoritmi che verranno illustrati in seguito, K-means clustering, Gaussian Mixture Models, Principal component analysis, sono algoritmi di apprendimento non supervisionato, mentre metodi come Linear regression, Logistic regression, K-nearest neighbor, Support vector machines e Random forest sono riconducibili ad apprendimento supervisionato.

1.1 Tensorflow

La libreria open-source Tensorflow, rilasciata nel 2015, è dedicata alla costruzione di reti neurali e ha ricoperto un ruolo importante nell'evoluzione del settore. Ad oggi è la libreria che, insieme a Pytorch [24], è diventata lo standard *de facto* per lo sviluppo di modelli ML ed in particolare DL. Attraverso un unico grafico di flusso (*dataflow graph*) viene rappresentata l'intera computazione di un modello, infatti sono compresi tutti i calcoli, gli stati, i parametri, le regole di aggiornamento degli stessi e le modalità di pre-processing dei dati. I componenti del grafico sono: *Tensors*, ovvero array con n dimensioni; *Operations* che sono azioni di trasformazione sui Tensors; *Variables*, operazioni con un buffer variabile; diverse implementazioni di *Queues* composte da Tensors [1]. Con le API è possibile gestire lo sviluppo del modello, il monitoraggio delle performance e il debugging, oltre alla fase di "serving" in cui si fornisce un modello allenato pronto all'uso da parte di terzi. Fra i vantaggi offerti da Tensorflow ci sono la possibilità di creare modelli complessi, combinando algoritmi

già esistenti e offerti direttamente dalla libreria, oltre alla semplicità con cui si può ottimizzare il codice per l'esecuzione su hardware specifico. Un altro aspetto fondamentale è quello di poter sfruttare l'efficienza delle API offerte per lo streaming dei dati in pipeline di modelli DL.

Il modulo Keras offre API di alto livello che offrono le componenti necessarie a costruire reti neurali complesse. Le tre API principali sono:

- Sequential, per la costruzione di reti semplici, ovvero con un singolo input, un unico flusso attraverso gli hidden layers e un output.
- Functional consente di formare reti più complesse, con molteplici input, hidden layers paralleli e più di un output.
- Con Sub-classing è possibile personalizzare ogni singolo layer della rete, dunque si può fare in modo che ognuno sia un sotto-modello del modello finale.

All'interno di `tf.keras.layers` si trovano i layer già implementati, i quali consistono in una funzione di computazione che opera con Tensors, e alcune variabili che rappresentano i pesi del singolo componente. Grazie a tali variabili, un layer mantiene il proprio stato, dunque ogni volta che la funzione al suo interno viene chiamata il punto di partenza della computazione dipende dal risultato del ciclo precedente. L'API in questione consente una gestione dettagliata di ciascun layer, permettendo di impostare la tipologia, la forma dell'input e dell'output, la funzione di attivazione, il numero di nodi che lo compongono. Qualora si realizzi una rete con Sub-classing, si definiscono uno o più layer personalizzati, estendendo la classe Layer e ridefinendo le funzioni `__init__()`, `build()` e `call()`. La prima serve per l'inizializzazione del layer, la seconda per la creazione dei parametri del modello, la terza è la computazione da svolgere con i Tensors che il layer riceve in input.

Una volta che si è definito il codice per l'addestramento del modello, attraverso Tensorboard si può monitorare il processo di training, in questo modo è possibile individuare eventuali anomalie. Inoltre, il servizio permette di visualizzare i dati e verificare le performance del modello. Il monitoraggio è fondamentale soprattutto per impedire overfitting o underfitting.

2 Apache Spark

Spark è una tecnologia open-source per l'elaborazione dei dati su cluster di computer, supporta una varietà di linguaggi di programmazione comuni e include librerie per molti compiti diversi. Il suo scopo è rendere semplice l'elaborazione di Big Data.

Architettura. L'architettura di Spark si basa su due astrazioni principali: Resilient Distributed Datasets (RDD), per l'archiviazione dei dati, e Directed Acyclic Graph (DAG), per l'elaborazione dei dati. Con "Resilient Distributed Datasets" si indicano le unità fondamentali con cui vengono gestiti i dati, sono collezioni di elementi che possono essere distribuiti tra nodi in un cluster e su cui è possibile lavorare in parallelo. In caso di un guasto, con gli RDD è possibile ricalcolare i dati: agiscono come un'interfaccia verso i dati. Una volta che i dati sono caricati in un RDD, Spark esegue trasformazioni e azioni sugli RDD in memoria (Spark memorizza anche i dati in memoria a meno che il sistema non la esaurisca o l'utente decida di scrivere i dati su disco per la persistenza) e distribuisce automaticamente le operazioni. Ogni trasformazione è un insieme di operazioni applicate per creare un nuovo RDD. Ogni volta che un nodo Spark riceve come compito lo svolgimento di operazioni, il suo ruolo è quello di applicare il calcolo agli RDD e restituire il risultato al driver. "Directed Acyclic Graph" è l'espressione con cui si indica il livello di pianificazione dei lavori in Spark, implementa un modello di esecuzione *stage-oriented* e quindi elimina il modello di esecuzione Hadoop MapReduce. Per esecuzione *stage-oriented* si intende che ogni lavoro eseguito da Spark viene diviso in insiemi più piccoli di calcoli (detti appunto *stage*) e vengono eseguiti singolarmente, anche se dipendono l'uno dall'altro. In sintesi, DAG coordina i nodi worker nel cluster e rende possibile la risoluzione degli errori. I tipi di dati con cui lavora Spark sono DataFrames e DataSets. I DataFrames sono degli strumenti per contenere dati e offrono delle API per la distribuzione su più nodi. Sono simili a una tabella di un database relazionale o a un file csv con intestazione: i dati sono organizzati in righe e colonne (anche di tipi eterogenei) e l'elaborazione viene eseguita attraverso funzioni definite dall'utente e altre funzioni di manipolazione (group, join, sort), sono un'astrazione implementata nel modulo Spark SQL. I DataSets sono una collezione di oggetti JVM fortemente tipizzati, quindi sono un'interfaccia di programmazione orientata agli oggetti sicura dal punto di vista del tipo, a differenza dei DataFrames che forniscono uniformità tra più linguaggi.

Componenti. L'architettura è composta da quattro componenti: il driver, gli esecutori, il cluster manager e i nodi worker. Lo Spark driver è il nodo principale che controlla il cluster manager, il quale a sua volta gestisce i nodi worker (anche detti slave) e fornisce i risultati dei dati al client dell'applicazione. Quando viene eseguito, il driver chiama l'applicazione reale e costruisce lo SparkContext che contiene tutte le funzioni di base per il funzionamento dell'applicazione. Il driver e il cluster manager controllano l'esecuzione dei nodi worker: il cluster manager assegna le risorse per i lavori da svolgere, quindi una volta che il lavoro è stato suddiviso in componenti più piccoli e distribuito

ai nodi worker, il driver controlla l'esecuzione. Molte tipologie di worker elaborano un RDD creato nel contesto Spark, quindi i risultati di ciascun nodo possono essere memorizzati nella cache [25]. In sintesi:

- il cluster manager assegna le risorse necessarie per l'elaborazione dei dati, ci sono diversi gestori fra cui YARN, Mesos e Kubernetes;
- il driver crea lo `SparkContext` nell'applicazione. Non appena il lavoro Spark viene avviato, esegue il metodo `main()` dell'applicazione e crea il DAG che rappresenta il flusso interno dei dati. In base ad esso, il driver richiede al cluster manager di allocare le risorse (nodi worker) necessarie per l'elaborazione. Una volta allocate le risorse, il driver utilizza lo `SparkContext` per inviare il codice unito ai dati verso i nodi worker per eseguirlo come Task, e ne riceve il risultato;
- un nodo worker è un componente del cluster che gestisce le risorse e ospita gli esecutori. È responsabile del coordinamento e della supervisione dell'esecuzione dei compiti su quel particolare nodo, comunica con il cluster manager per ottenere le risorse e ne segnala l'utilizzo. Implementa l'ambiente di esecuzione per gli esecutori, gestendo il loro ciclo di vita sul nodo;
- un esecutore è un processo in esecuzione su un nodo worker in un cluster. Esegue i compiti assegnati dal driver Spark ed è responsabile per l'esecuzione della sua unità di lavoro. Per impostazione predefinita, Spark crea un esecutore per ogni nodo nel cluster, ma è possibile configurarne il numero in base alle esigenze dell'applicazione. Ci sono quattro tipi di esecutori: default, per compiti di elaborazione dati generici; coarse-grained, per compiti che richiedono più memoria; fine-grained, per compiti che richiedono meno memoria; external executors, utilizzati quando l'applicazione deve usare risorse esterne per l'elaborazione (ad esempio una GPU per l'elaborazione).

Esecuzione. La classe `SparkSession` permette di accedere a tutte le funzionalità di Spark: incapsula `SparkConf` e `SparkContext`, è creata con il builder design pattern e una volta istanziata è possibile configurare le proprietà dell'ambiente di esecuzione di Spark. A tal proposito, si distinguono tre diversi modi di esecuzione:

1. Local è l'impostazione ideale per il testing, consiste nell'eseguire Spark su una singola macchina, tutta l'elaborazione è eseguita da un singolo processore.

2. Con cluster un jar precompilato (o uno script python/R) viene consegnato a un cluster manager, poi il driver viene avviato su un nodo worker all'interno del cluster, oltre ai processi degli esecutori.
3. Utilizzando client il programma driver viene eseguito sulla macchina client, mentre gli esecutori vengono eseguiti sul cluster.

I possibili cluster manager sono Standalone, Apache Mesos, Hadoop Yarn e Kubernetes. Il primo è incluso nelle distribuzioni Spark e permette una gestione molto basilare di un cluster, senza fornire funzionalità di automazione. Mesos è deprecato e quindi non più usato a partire da Spark 3.2.0. Il terzo è il resource manager per Hadoop 3. Kubernetes è l'orchestratore utilizzato in questa tesi, consente una gestione dettagliata del cluster, come illustrato nella sezione 4.

Object storage. L'object storage è un metodo di archiviazione dei dati con il quale si organizzano le informazioni in "oggetti". Ogni oggetto contiene i dati stessi, insieme a un identificativo univoco e ai metadati, come ad esempio il formato del contenuto, la data di creazione e le autorizzazioni di accesso. Gli oggetti vengono memorizzati in un ambiente distribuito, quindi è facile scalare la capacità di archiviazione aggiungendo nuovi nodi al sistema, specialmente se si utilizza un servizio in cloud. Questo approccio è particolarmente adatto per applicazioni che richiedono un accesso rapido e affidabile a una grande quantità di dati non strutturati, come gli algoritmi ML e DL. L'object storage offre inoltre una maggiore resistenza agli errori rispetto ad altri metodi di archiviazione, in quanto i dati vengono replicati su più nodi per garantire la disponibilità continua anche in caso di guasti hardware. Il servizio Amazon Simple Storage Service (S3) è un sistema di object storage in cloud. I dati vengono archiviati come oggetti in entità dette bucket, distribuendo delle repliche all'interno di server distribuiti su un'ampia regione geografica. Offre diverse funzionalità di sicurezza, tra cui ACL, crittografia lato server e policy personalizzate per ogni bucket, in modo tale da proteggere i dati da accessi non autorizzati. Le politiche di accesso possono essere definite direttamente sul bucket in formato JSON, oppure si possono utilizzare le policy IAM per un controllo degli accessi basato su ruoli. Un'ulteriore funzionalità offerta è quella del versioning per la memorizzazione delle modifiche. Il servizio è altamente scalabile, gestisce grandi quantità di dati e traffico, inoltre è facilmente integrabile in sistemi esistenti grazie alle API offerte.

MLlib. La libreria Spark MLlib serve per il training e il tuning di diversi algoritmi ML, in maniera distribuita e con interfacce indipendenti dall'algoritmo sottostante. Implementa le astrazioni di *estimators*, *transformers*, *evaluators* e

offre, attraverso opportune API, numerosi algoritmi ML. I dati sono gestiti attraverso gli oggetti `DataFrame`. I transformers sono impiegati per modificare i dataset aggiungendo previsioni, dunque i modelli ML in `MLlib` sono a tutti gli effetti transformers. Gli estimators generano transformers adattandoli ai dati, attraverso il metodo `fit()` si usano gli estimators per allenare un modello su un dataset. Gli evaluators analizzano le prestazioni del modello. La libreria offre anche la possibilità di unire transformers ed estimators a formare oggetti di tipo `PipelineStages`, in modo tale da definire una sequenza ordinata di operazioni da svolgere (Pipeline) al fine di realizzare un flusso di lavoro completo che porta dai dati grezzi al modello ML finale. Ogni componente della sequenza riceve in ingresso un `DataFrame` e ne produce uno in output, chiamando il metodo `fit()` sull'oggetto Pipeline si avvia l'esecuzione di tutti i passaggi [38].

3 Docker

Docker è uno strumento di virtualizzazione che, a differenza delle macchine virtuali —Virtual Machines (VM)— si occupa di virtualizzare solo il livello applicativo del sistema operativo. Ogni unità di virtualizzazione è chiamata *Container* ed esegue un'immagine che contiene una versione minimale di un sistema operativo; tutti i Container in esecuzione su una macchina condividono il kernel della macchina host. La gestione dei file persistenti avviene utilizzando i volumi: unità di memorizzazione che vengono montate sui Container. I vantaggi offerti da questo approccio riguardano soprattutto la possibilità di avere immagini molto leggere, quindi, a parità di risorse, è possibile eseguire più Container rispetto a VM. Una diretta conseguenza di ciò è la velocità con cui è possibile avviare un sistema di Container. Questa tecnologia è stata sviluppata per facilitare lo sviluppo e il deployment di applicazioni, infatti ogni singolo Container può essere creato in modo tale che contenga tutto il necessario per l'esecuzione di una certa applicazione, disaccoppiando il software che si sviluppa dall'hardware della macchina host. Un aspetto centrale di Docker è l'efficientamento delle risorse: idealmente ogni Container ha solo lo stretto necessario per l'esecuzione del codice a cui è dedicato, non virtualizza nulla di più.

Una caratteristica fondamentale per l'isolamento fra diversi Container è la separazione dei *PID namespaces* fra le diverse unità di virtualizzazione. Come noto, ogni processo in ambiente Linux è identificato da un PID, e dal momento che tutti i Container in esecuzione su una certa macchina condividono le stesse risorse, per evitare conflitti fra processi è fondamentale che ogni Container abbia il proprio PID namespace. Questo significa che il processo con $PID = 1$

nel Container A non è il processo con $PID = 1$ nel Container B (anche se potrebbero essere lo stesso programma).

Il software al cuore di Docker è Docker Engine ed è pensato per utilizzare il kernel Linux, quindi non è possibile eseguirlo direttamente su una macchina Windows o MacOS, ma è disponibile l'applicativo Docker Desktop che crea una macchina virtuale Linux, molto leggera, su cui poi vengono eseguiti i Container. Docker Engine è un'applicazione client-server composta dal demone `dockerd` (server), dalle API per dare comandi al demone e da un'interfaccia da riga di comando (client). Il demone è il componente che si occupa nel dettaglio di amministrare i singoli Container, caricando le immagini e avviando i processi in base ai comandi che vengono impartiti dal client.

La configurazione di un singolo Container avviene tramite la scrittura di un *Dockerfile*, ossia un file testuale in cui vengono definite le risorse che devono essere caricate nell'immagine, i comandi che devono essere eseguiti, le porte che il Container deve esporre verso la rete, i volumi a cui deve avere accesso. Per la creazione dell'immagine si parte da una di base fra quelle disponibili in uno dei registry, ossia repository di immagini. In particolare, è possibile avere un servizio di distribuzione delle immagini locale, oppure utilizzare una repository pubblica, la più comune delle quali è Docker Hub.

Per rendere più agevole la configurazione di molteplici Container, è disponibile il tool *Docker Compose*. Per utilizzare questo strumento bisogna scrivere un file in formato YAML in cui si definisce quanti Container creare, seguendo quali Dockerfile ed eventualmente quali comandi eseguire. L'utilizzo di Docker Compose è utile in particolare quando è necessario configurare una rete di Container.

Una peculiarità dei Container Docker è quella di poter essere usati per offrire microservizi —accessibili in rete da parte di utenti o altri software— semplificando molto la loro amministrazione. Nel momento in cui ogni servizio è semplicemente un Container in esecuzione, infatti, è molto semplice e veloce aggiornarlo o riavviarlo in caso di errore, inoltre la configurazione di rete può essere realizzata e modificata unicamente via software. Per assistere gli sviluppatori nella gestione di servizi composti da un elevato numero di Container, sono stati sviluppati degli orchestratori [6] fra i quali è emerso Kubernetes.

4 Kubernetes

L'orchestratore Kubernetes serve per amministrare grandi sistemi software di microservizi, automatizzando buona parte della gestione e così migliorando l'affidabilità e la scalabilità dei sistemi. Le risorse gestite con Kubernetes sono organizzate in un cluster, ossia un insieme di nodi, fisici o virtuali, che eseguo-

no diversi processi. Su ogni nodo vengono eseguiti uno o più *Pod* che sono un'astrazione dei Container. Kubernetes garantisce load-balancing sui Container esposti in rete, inoltre gestisce in maniera automatica e ottimizzata i sistemi di storage. I Container che incorrono in un errore vengono automaticamente riavviati, e in generale lo stato del cluster viene mantenuto come da configurazione. Le risorse possono essere isolate in gruppi all'interno di uno stesso cluster utilizzando i namespace. I nomi devono essere unici all'interno dello stesso namespace, ma non fra namespace diversi.

Al cuore del funzionamento di Kubernetes c'è il *Control Plane*, un nodo su cui sono in esecuzione i principali processi di gestione di un cluster. Il primo è l'API Server che è l'interfaccia principale per l'accesso al cluster e in particolare è il componente con cui la web User Interface (UI), i comandi della Command Line Interface (CLI) e le API dei linguaggi di scripting si interfacciano. Il Controller Manager si occupa di controllare la composizione del cluster, dunque è fondamentale, per esempio, per avviare un nuovo Container nel momento in cui uno in esecuzione dovesse incorrere in un errore. Lo Scheduler si occupa di decidere su quale nodo worker eseguire un Container in base al carico computazionale che ciascun nodo sta sopportando. Lo storage etcd contiene lo stato corrente del cluster Kubernetes, fondamentale per la disaster recovery.

Il Control Plane comunica con i nodi worker attraverso la Virtual Network interna al cluster: ogni Pod, al momento della creazione, ottiene un indirizzo IP tramite cui è raggiungibile dagli altri Pod. Un Pod può facilmente terminare la propria esecuzione, per esempio se l'applicazione al suo interno incorre in un errore o termina le risorse. Dal momento che ogni volta che un Pod viene riavviato cambia indirizzo IP, se si vuole che un componente sia sempre raggiungibile dagli altri occorre implementare i Kubernetes Service. Tramite un Service si assegna ad un determinato Pod un indirizzo statico, un Service può essere interno (raggiungibile solo da nodi interni al cluster), oppure esterno (raggiungibile anche dalla rete pubblica). Per associare un nome di dominio a un Service si utilizzano gli Ingress.

I dati di configurazione necessari alle applicazioni in esecuzione nei Pod sono scritti nei ConfigMap (dati non sensibili, come Uniform Resource Locator (URL) di database) e Secret (dati sensibili, come credenziali e certificati) in modo tale che possano essere cambiati in modo agevole, senza bisogno di modificare e ricompilare il codice delle applicazioni.

I dati persistenti sono memorizzati nei volumi, i quali possono essere locali, ovvero su un nodo del cluster, o remoti. In entrambi i casi il volume non è gestito da Kubernetes, quindi la creazione e la formattazione di tali unità di archiviazione va fatta manualmente.

Su tutti i nodi diversi dal Control Plane sono in esecuzione kubelet e kube-proxy. Il primo si assicura che i Container configurati siano in esecuzione all'in-

terno dei Pod, mentre il secondo controlla le regole relative alla comunicazione in rete, così da rendere concrete le configurazioni dei Service. Al momento della creazione di un Service, l'API Server gli assegna un IP virtuale e lo notifica al kube-proxy. Questo, utilizzando iptables, rende indirizzabile tale IP all'interno del nodo in cui è in esecuzione. Un ulteriore processo presente sui nodi worker è CoreDNS: un server DNS che serve per la risoluzione interna dei nomi di dominio e può essere utilizzato per la procedura di indirizzamento dei Service.

Un ulteriore livello di astrazione rispetto ai Pod è rappresentato da Deployment e StatefulSet. Per gestire in modo automatico lo scaling delle risorse e l'avvio di nuovi Pod in caso di errore, si configura il cluster utilizzando Deployment per i nodi che non hanno uno stato e StatefulSet per i nodi che lo hanno, come i database.

La configurazione di ogni elemento avviene in maniera dichiarativa, sottomettendo all'API Server dei file in formato YAML che riportano le specifiche desiderate. La responsabilità del Controller Manager è quella di mantenere sempre attive le risorse specificate nei file di configurazione. I file relativi ai componenti del cluster hanno una prima riga in cui è indicata la versione dell'API che si utilizza, una seconda riga in cui si definisce il tipo di componente (Pod, Deployment, ...). In seguito è definita una sezione dedicata ai metadati del componente, e una in cui si definisce lo stato desiderato, ovvero, per esempio, il numero di repliche di un Pod.

Per avviare un cluster Kubernetes in locale, si può utilizzare lo strumento open-source minikube che consente di gestire un semplice cluster su una singola macchina. In questo lavoro si è deciso invece di usare il server Kubernetes integrato in Docker Desktop che permette di utilizzare un cluster a singolo nodo. Per interfacciarsi con Kubernetes si è utilizzato lo strumento kubectl che permette di inviare comandi dalla CLI all'API Server di Kubernetes.

Spark su Kubernetes. A partire dalla versione di Spark 2.3 è possibile usare Kubernetes come scheduler per uno Spark cluster. L'amministrazione del sistema tramite Kubernetes, come visto, offre una grande possibilità di automazione e configurazione avanzata. Grazie all'uso di un orchestratore avanzato è possibile sviluppare pipeline per l'elaborazione dati molto avanzate ed eseguire in maniera semplice diversi algoritmi ML all'interno del cluster. Un secondo vantaggio fondamentale è quello di poter cambiare facilmente l'infrastruttura fisica sottostante al cluster. Come si dimostra in questo lavoro, infatti, è possibile passare da un cluster in esecuzione su Platform as a Service fornita da Amazon Web Services (AWS) ad uno come quello presente nell'infrastruttura MUSA, senza ripensare la configurazione di deployment.

Capitolo 3

Descrizione del lavoro svolto

In questo capitolo si vuole offrire una panoramica del lavoro di tesi. Come illustrato nel capitolo 1, l'attività ha previsto il recupero di algoritmi ML esistenti sviluppati con Java/Spark e, parallelamente, lo studio di una modalità per l'implementazione di nuovi modelli DL con la libreria Tensorflow. Il requisito richiesto ad entrambe le attività è quello di arrivare ad avere dei modelli per l'analisi dati eseguibili in maniera distribuita su un cluster basato su Kubernetes.

Nella fase preliminare si sono analizzati gli algoritmi Java già implementati e selezionati i più interessanti ai fini dell'analisi dati, e sono state analizzate le dipendenze di ciascun modello. In seguito è stato svolto l'aggiornamento del codice di questi algoritmi, provvedendo a risolvere i problemi di compatibilità emersi dall'aggiornamento. Per quanto riguarda, invece, il modello in Tensorflow, si è scelto di sviluppare una semplice rete Convolutional Neural Network (CNN).

Al fine di arrivare alla distribuzione con Kubernetes, sia nell'attività di recupero sia in quella di sviluppo si è dovuto rendere il codice eseguibile all'interno di Container Docker, costruendo le relative immagini e risolvendo problemi di compatibilità ove necessario. Dunque, a questo punto del lavoro si sono creati dei Container Docker in cui sono stati eseguiti gli algoritmi, inserendo gli archivi JAR per i modelli Java/Spark e direttamente il codice python per quello in Tensorflow.

Una volta testata l'esecuzione in Container, si è passati a fare un test di esecuzione su cluster Kubernetes locale, ovvero virtualizzato su un'unica macchina. Questa fase del lavoro, quindi, ha previsto la configurazione del cluster e l'esecuzione degli algoritmi Java/Spark. Si sono studiate le modalità di distribuzione del calcolo computazionale attraverso la libreria Tensorflow, poi —sfruttando il cluster locale configurato e utilizzato per gli algoritmi Java/Spark— si è eseguito il training distribuito del modello Tensorflow. Dal momento che l'infrastruttura di esecuzione è soltanto locale alla macchina, non si è utilizzato nessun siste-

ma di storage distribuito, ma si sono semplicemente usate le immagini Docker costruite nella fase precedente per avviare i Container del cluster.

Il passaggio successivo è stato svolto con l'obiettivo di fare un'ulteriore prova di esecuzione, avvicinandosi all'architettura dell'ambiente distribuito in cui era previsto il deployment finale. In questa fase, dunque, è stato configurato un cluster Kubernetes sui server forniti da AWS, utilizzando il servizio Elastic Kubernetes Service (EKS). In questo modo è stato possibile distribuire il carico computazionale realmente, ovvero utilizzando diversi processori (a differenza di quanto avveniva nella fase precedente, con cluster locale). Un'ulteriore caratteristica sviluppata in questa fase è stata la possibilità di utilizzare un'immagine Docker generica, ossia senza codice caricato in locale, perché si è sfruttato il sistema di storage distribuito Amazon Simple Storage Service (S3) per leggere l'archivio JAR (per i modelli Java/Spark) e il codice python (per il modello Tensorflow). Con questo passaggio, quindi, si è illustrato un metodo per allenare ed eseguire i modelli ML in maniera distribuita superando la necessità di ricostruire le immagini Docker ad ogni modifica del codice.

Avendo eseguito con successo i task sul cluster, si sono analizzati i tempi e i log di esecuzione per accertare l'effettiva distribuzione del carico computazionale. Nello specifico, si è accertato che nell'esecuzione dei task Java/Spark venissero richieste le risorse corrette e fossero istanziati sia lo Spark driver, sia i nodi worker. Inoltre si è accertato che il driver suddividesse il carico fra i diversi worker e la comunicazione fra i vari componenti del cluster avvenisse correttamente, portando ad una corretta esecuzione e alla terminazione dei processi in esecuzione sui diversi nodi. Per il modello Tensorflow, invece, si è verificato che tutti i nodi configurati si collegassero fra loro utilizzando il protocollo desiderato. In aggiunta si è accertato che il nodo chief riuscisse a ricevere gli aggiornamenti delle variabili del modello da parte di tutti gli altri componenti del cluster, così che il training venisse portato a termine correttamente.

La fase finale si è concentrata sul deployment sull'infrastruttura sviluppata nell'ambito del progetto MUSA. La prima operazione è stata la configurazione dello strumento per interagire con il cluster e delle impostazioni di rete della macchina, in modo tale da potersi connettere ai server dedicati. In seguito si è analizzata l'architettura del cluster in modo tale da comprendere il metodo migliore per l'esecuzione dei task, e soprattutto come configurare lo strumento per l'avvio dell'esecuzione per avere i permessi necessari. Una volta individuato il Service Account corretto (maggiori dettagli nella sezione 2 del capitolo 7) e configurato un Application-level gateway per inoltrare al cluster il traffico diretto verso localhost, si sono eseguiti i modelli ML con successo. Anche in questo caso sono stati analizzati i dati di log per accertarsi che venisse seguita la sequenza di operazioni attesa per la distribuzione del carico computazionale.

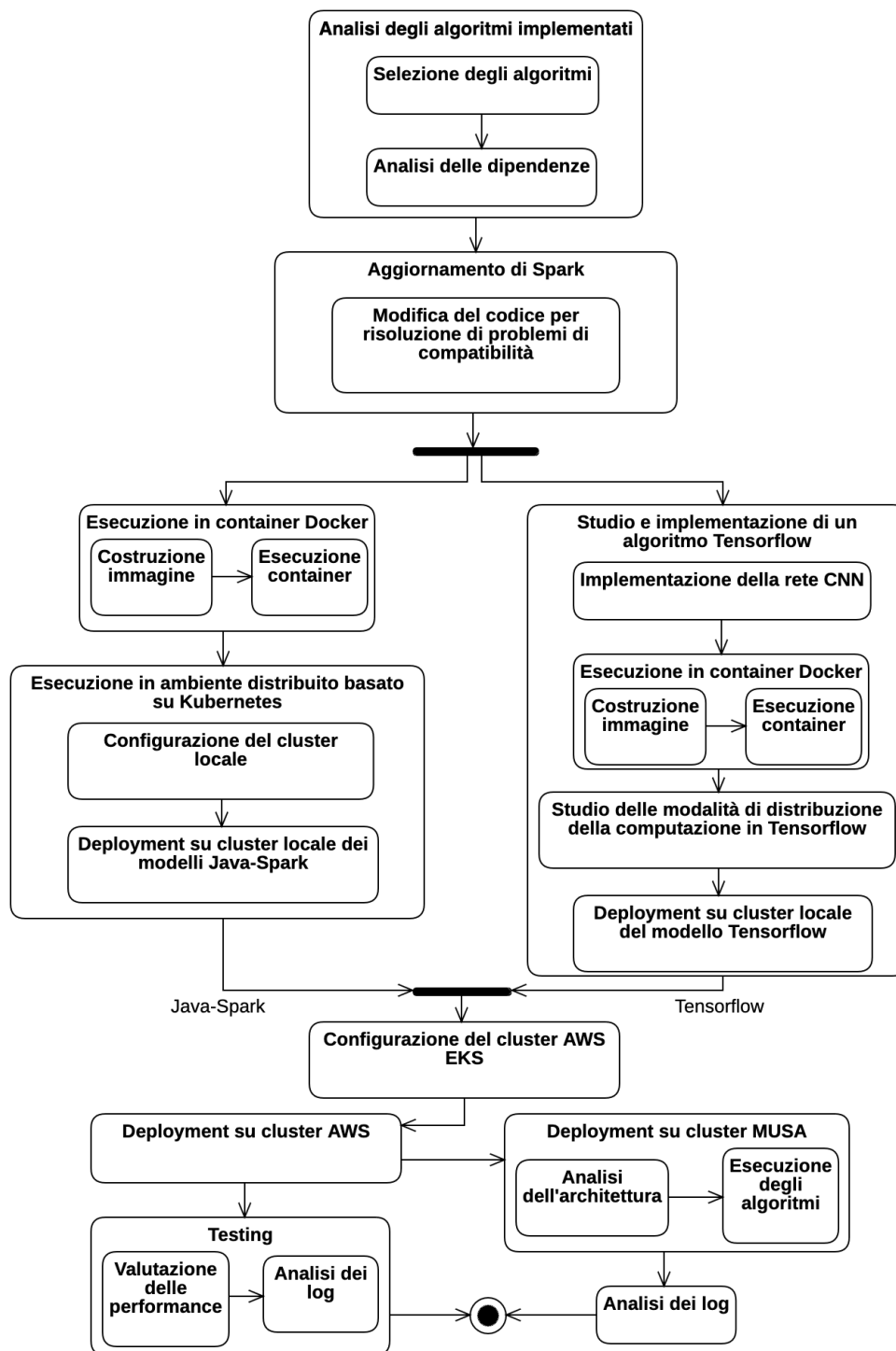


Figura 3.1 : Flusso di lavoro

Capitolo 4

Scenario di partenza

La prima fase del lavoro si è concentrata sul recupero di alcuni algoritmi sviluppati con tecnologia Spark sfruttando la libreria MLlib, originariamente implementati nell'ambito del progetto EVOTION [3]. Fra i task sviluppati, sono stati selezionati quelli che implementano le tecniche di inferenza e gli algoritmi illustrati di seguito.

K-means clustering. K-means è un algoritmo di clustering con approccio partizionale, ottiene in input il parametro k , numero di cluster, poi la computazione inizia selezionando k centroidi casuali dal dataset su cui è eseguito, viene calcolata la distanza fra ogni punto del dataset e i centroidi, in modo da assegnare ogni dato al cluster relativo al centroide più vicino. Ogni volta che un nuovo punto viene assegnato al cluster, il centro viene ricalcolato, quindi l'algoritmo viene eseguito iterativamente finché i cluster non sono stabili (nessun punto viene più assegnato a un nuovo cluster) [19]. Osservando il clustering come un *learning problem* —ovvero trattandolo come un compito di apprendimento automatico in cui l'obiettivo è assegnare automaticamente gli oggetti in gruppi omogenei— si utilizza l'algoritmo K-means per allenare un modello ML che possa essere sfruttato per poi svolgere delle predizioni. In questo caso, fornito un nuovo punto che rappresenta un dato non presente nel dataset con cui si è svolto il training, il modello deve essere in grado di collocarlo in uno dei cluster che ha individuato. Fra i task implementati, uno è dedicato alla creazione del modello e uno all'utilizzo di tale modello per svolgere le predizioni; è inoltre presente un task dedicato alla valutazione del modello, sfruttando il *Silhouette score* [32]. Nella versione classica dell'algoritmo la metrica utilizzata è la distanza euclidea, ma sono state proposte diverse alternative al fine di migliorare le performance [9, 8, 33]. La variante bisecting K-means [35] sfrutta l'algoritmo di base per implementare una procedura differente. Inizialmente il dataset è considerato come un unico cluster e viene utilizzato K-means per dividerlo in

due. Si ripete il processo un certo numero di volte (parametro dell'algoritmo) e poi si tiene la sezione che presenta due cluster con la più alta similarità fra gli elementi. Infine vengono reiterati i passaggi precedenti fino a raggiungere il numero di cluster desiderato. Come nel caso dell'algoritmo di base, anche bisecting K-means viene utilizzato per creare un modello con cui poi effettuare delle predizioni.

Decision Tree Classifier. I classificatori ad albero sono modelli predittivi con una struttura assimilabile a un diagramma di flusso, ideali per il data mining supervisionato. Ogni nodo dell'albero è un test su un attributo del dato da classificare, ogni ramo è un possibile esito del test e ogni foglia indica l'etichetta di classificazione. Il nodo radice non ha rami in ingresso ed è infatti il punto di ingresso all'albero stesso. La classificazione viene effettuata eseguendo i test rappresentati dai nodi, di volta in volta scegliendo il test successivo sulla base del risultato del precedente, continuando fino all'ultimo livello (terminale), in cui tutte le tuple di dati in un nodo comprendono campioni di una sola classe. La costruzione del modello avviene seguendo la fase di generazione (*generation*) prima e di potatura (*pruning*) poi. Nella prima fase tutti i dati di allenamento partono nel nodo radice, viene definito un criterio di divisione ed è selezionato il miglior attributo per effettuare la divisione. In seguito i dati vengono partizionati nei nodi figli e la procedura è ripetuta fino a quando aggiungendo nodi foglia non si aumenta più la precisione della classificazione. Nella seconda fase si punta a ridurre le dimensioni del modello in modo da renderlo più efficiente, in particolare vengono rimossi i sotto-alberi che si sono creati per riflettere le caratteristiche di eventuale rumore nei dati o outliers [26]. Una volta che si ha un modello affidabile ed efficiente, lo si può utilizzare per la classificazione di dati.

Gaussian Mixture Model. Si tratta di un modello statistico utilizzato per rappresentare la distribuzione di probabilità di un insieme di dati come una combinazione di più distribuzioni gaussiane. Un Gaussian Mixture Model (GMM) assume che i dati siano generati da un insieme di sotto-popolazioni, ognuna che segue una distribuzione gaussiana. È definito da un numero specifico di componenti gaussiane, ognuna con la propria media e varianza; l'obiettivo dell'algoritmo è quello di costruire un modello sulla base di un certo insieme di dati, in modo tale da eseguire poi predizioni collocando nuovi dati nella gaussiana corretta (ovvero stimando la variabile latente) [27]. Il modello viene costruito utilizzando la classe offerta da Spark GaussianMixtureModel in un task, poi un secondo utilizza il modello per eseguire le predizioni.

K-Nearest Neighbors Classifier. L'algoritmo k-nearest neighbors [11] è stato utilizzato per costruire un classificatore, quindi il task che lo implementa ha l'obiettivo di restituire in output la classe a cui appartiene un oggetto che riceve in input. La classificazione avviene scegliendo la classe più comune fra i k oggetti con features più simili (e quindi "vicini") all'oggetto da classificare, dove k è parametro dell'algoritmo. Dal momento che il funzionamento dell'algoritmo si basa sul confronto della distanza fra l'oggetto da classificare e una serie di oggetti conosciuti, la fase di allenamento del modello consiste nell'estrazione delle caratteristiche dai dati dedicati al training, in modo da rappresentare ogni oggetto come un vettore in uno spazio n -dimensionale (dove n è il numero delle caratteristiche). Ogni vettore è in seguito assegnato a una classe sulla base della classificazione indicata nel dataset. Si tratta, dunque, di un metodo di apprendimento supervisionato e non parametrico [34]: non vengono fatte assunzioni sulla distribuzione dei dati su cui opera l'algoritmo, rendendolo adatto alla classificazione di dati di cui non si conosce la distribuzione. Occorre tenere presente, tuttavia, che il meccanismo con cui avviene la classificazione comprende intrinsecamente il rischio che la classificazione sia svolta con bassa accuratezza nel caso di distribuzione fortemente asimmetrica, perché gli esempi di una classe più frequente tendono a dominare la previsione di un nuovo esempio (tendono ad essere comuni tra i k vicini più prossimi a causa del loro grande numero), per questo sono state proposte delle alternative alla regola di base del voto a maggioranza dei k vicini [10]. Utilizzando le classi di Spark dedicate al modello KNNC —`KNNClassificationModel` e `KNNClassifier`— viene costruito il classificatore sulla base di un dataset in input, per poi utilizzare il modello per effettuare le predizioni.

Linear Discriminant Analysis. Si tratta un algoritmo di riduzione della dimensionalità e di classificazione utilizzato nell'ambito dell'apprendimento supervisionato. L'obiettivo è trovare la combinazione lineare delle caratteristiche che massimizza la separazione tra le classi dei dati utilizzati nell'allenamento del modello, per farlo l'algoritmo LDA proietta i dati in uno spazio delle caratteristiche a dimensioni inferiori, senza perdere informazioni, in modo che le diverse classi siano il più separabili possibile lungo questa nuova dimensione. Vengono calcolate le medie delle caratteristiche per ciascuna classe, oltre alle matrici di dispersione intra-classe (*data spread* all'interno di una classe) e inter-classe (*data spread* fra le classi). Successivamente si calcolano le direzioni discriminanti, ovvero i vettori che massimizzano il rapporto tra la varianza inter-classe e la varianza intra-classe, i dati vengono proiettati su queste direzioni per ridurre la dimensionalità e separare le classi nel nuovo spazio delle caratteristiche. Dopo la proiezione dei dati, è possibile utilizzare un classifica-

tore per i nuovi dati in base alla loro posizione nello spazio delle caratteristiche ridotto. LDA è spesso utilizzato come tecnica di riduzione della dimensionalità prima di applicare un classificatore, poiché aiuta a migliorare le prestazioni del modello riducendo il rischio di overfitting. È da notare che vengono fatte le assunzioni che i dati siano distribuiti normalmente e che le classi abbiano matrici simili [36]. Nel task implementato vengono utilizzate le classi di clustering offerte da Spark LDA e LDAModel per allenare il modello ed effettuare le predizioni.

Principal Component Analysis. In modo simile a LDA, questo algoritmo mira a ridurre la dimensionalità dei dati ed è quindi utilizzato nel *pre-processing*. L'idea di base è quella di estrarre le informazioni importanti dal dataset, per poi rappresentarle come variabili ortogonali (le componenti principali) ottenute come combinazioni lineari delle variabili originali, in modo tale da poter poi distinguere graficamente i diversi cluster. Per estrarre la prima componente si ricerca quella che ha varianza maggiore, perché in questo modo si estrae la maggior quantità di informazione dal dataset, per la seconda si pone il vincolo di ortogonalità con la prima e si cerca nuovamente quella con varianza maggiore (tolta la prima), reiterando il processo un numero pari alle componenti che si desidera estrarre [2]. Più nel dettaglio, il processo consiste in una prima normalizzazione dei dati (sottraendo la media e dividendo per la deviazione standard), per poi calcolare la matrice di covarianza (σ) dei dati. In seguito si calcolano autovalori e autovettori di σ , vengono selezionate le componenti principali estraendo gli autovettori con i corrispondenti autovalori più grandi, infine si proiettano i dati normalizzati sulle componenti principali estratte per ridurre la dimensionalità. Grazie alle classi di Spark PCA e PCAModel, il task esegue il *fitting* del modello e poi restituisce un dataset con le informazioni relative alle componenti principali.

ANalysis Of VAriance (ANOVA). ANOVA è una tecnica di inferenza statistica utilizzata per analizzare la differenza fra le medie di più gruppi differenti di dati, con il fine di descrivere le relazioni presenti fra le variabili presenti nel dataset. È particolarmente utile per analizzare i dati ottenuti da diverse misurazioni svolte in seguito a diversi generici "trattamenti" eseguiti nel corso di un esperimento, perché consente di valutare la significanza statistica dei "trattamenti" confrontando i dati con quelli di un ipotetico gruppo di controllo. Per utilizzare ANOVA si assume che ogni gruppo su cui sono state effettuate le misurazioni segua una distribuzione normale, le varianze delle popolazioni di provenienza dei gruppi siano uguali fra loro, le osservazioni sui vari gruppi siano indipendenti. L'ipotesi nulla è che tutte le medie delle diverse popolazioni

considerate siano uguali, quindi che non ci sia una differenza significativa fra i diversi gruppi, mentre l'ipotesi alternativa è che almeno una media sia differente, con un certo livello di significanza. La versione più semplice dell'analisi è *one-way ANOVA*, in cui viene esaminata l'influenza di una singola variabile indipendente, ed è quella eseguita dal task nel caso in cui il dataset in input sia composto da sole due colonne (una per la variabile dipendente e una per quella indipendente). La versione *two-way ANOVA*, che analizza l'influenza di due differenti variabili indipendenti, è eseguita se il dataset ha tre colonne, la versione *multi-way ANOVA* serve per l'analisi della varianza provocata da più di due variabili indipendenti simultaneamente ed è eseguita se il dataset ha almeno quattro colonne. L'algoritmo restituisce una tabella contenente i gradi di libertà, la somma dei quadrati, la media dei quadrati, l'*f-ratio*, il *p-value*, il numero di celle del dataset, la somma e la media dei loro valori. L'*f-ratio* è il rapporto fra la varianza tra i diversi gruppi, e la varianza all'interno dei singoli gruppi. Il *p-value* è la probabilità che l'*f-ratio* sia maggiore o uguale del valore atteso, calcolato prendendo direttamente il valore della *F-distribution* per il valore di significanza α , considerando i gradi di libertà di numeratore e denominatore dell'*f-ratio*. Se *p-value* è minore di α l'ipotesi nulla è falsa, quindi c'è una differenza significativa dal punto di vista statistico fra le medie dei gruppi [30].

Linear Regression. La regressione lineare è una tecnica statistica utilizzata per modellare la relazione tra una variabile dipendente e una o più variabili indipendenti, in modo da definire la variabile dipendente come una combinazione lineare di alcuni valori (le variabili indipendenti), detti predittori. Una volta individuata la relazione fra tali valori e la variabile dipendente, è possibile effettuare delle predizioni sulla base dei valori osservati. La forma più comune è la regressione lineare semplice —Simple Linear Regression (SLR)— che coinvolge una sola variabile indipendente, ma esistono diverse varianti della regressione lineare che possono essere utilizzate in diverse situazioni. L'obiettivo di SLR è quello di trovare una funzione lineare che predica il valore della variabile dipendente dato un certo valore di quella indipendente (ovvero data l'ascissa, il valore dell'ordinata deve essere una predizione accurata): le predizioni sono svolte sulla base di un solo predittore [31]. Un Generalized Linear Model (GLM) è una generalizzazione che mira a rendere più flessibile la regressione lineare classica. Le variabili coinvolte possono seguire una distribuzione arbitraria, e viene utilizzata una *link function* che mette in relazione i predittori con la media della distribuzione della variabile dipendente. Aniché variare linearmente la variabile dipendente stessa, in questo modello è una funzione di tale variabile che varia linearmente con i predittori. La regressione lineare è

un GLM in cui la distribuzione della variabile dipendente è una normale con varianza costante e la *link function* è l'identità [14]. Il task implementato offre la possibilità di scegliere, con un opportuno parametro, se utilizzare la SLR o la Generalized Linear Regression (GLR) che utilizza un GLM.

Logistic Regression. Nel caso in cui si abbia a che fare con delle variabili che possono assumere soltanto due valori (circostanza frequente nei classificatori), è possibile utilizzare un particolare tipo di regressione —la regressione logistica— che utilizza la funzione logit per associare ad un vettore in ingresso un valore di probabilità di appartenenza ad una delle due classi possibili [14, 18]. Per costruire un modello ML, durante la fase di addestramento (supervisionato) si utilizzano i dati dedicati per trovare i coefficienti della funzione logit che massimizzano la corrispondenza fra i dati e la loro classe di appartenenza. In seguito viene stabilito il *decision boundary* per separare le diverse classi nello spazio delle features. Una volta allenato il modello, si può utilizzare per effettuare classificazione presentando in ingresso nuovi dati: l'output del modello è la probabilità che il vettore in ingresso rappresenti un oggetto appartenente a una delle due classi. I task implementati offrono anche la possibilità di vedere i parametri necessari alla valutazione del modello (precision, recall, f-score) e l'evoluzione della precisione del modello durante la fase di addestramento.

1 Analisi e operazioni preliminari

Affinché i singoli task Spark fossero eseguibili singolarmente, dopo una prima analisi del codice già sviluppato, si è resa necessaria l'estrazione delle singole classi Java che implementano i diversi algoritmi e la contestuale creazione di un progetto autonomo per ogni algoritmo. L'autonomia dei singoli progetti è un requisito necessario per avere la possibilità di creare in maniera semplice i cosiddetti *fat jar*, o *uber jar* (maggiori dettagli in seguito). Dal momento che le classi sono state estratte da un progetto più grande, è stato necessario risolvere le dipendenze perché ogni algoritmo potesse importare tutte le classi e librerie esterne di cui necessitava. Un'ulteriore attività svolta è stata quella di aggiornamento della versione di Spark dalla 2.2.0 alla 3.5.0, mantenendo Java 8. Numerosi algoritmi fra quelli coinvolti nel lavoro utilizzano la classe "VectorDisassembler", realizzata in Scala 2.11 per Spark 2.2.0 da un singolo sviluppatore e non più aggiornata, dunque è stata creata una *fork* della classe in modo da poter aggiornare la versione di Scala e così rendere il codice compatibile con Spark 3.5.0.

Struttura generale del codice. I file del codice relativo a ciascun task sono inseriti all'interno di un package Java dedicato, che riporta nella denominazione il nome dell'algoritmo implementato dal rispettivo task (`it.unimi.evotion.tasks.<nome_algoritmo>`). Ciascun algoritmo è composto di base da due classi principali: il codice che implementa l'algoritmo vero e proprio è contenuto in una classe che implementa l'interfaccia `Task`, definita per fare in modo che ogni algoritmo implementi i metodi `init`, `run` e `postProcessing`; l'altra classe è dedicata al *main* e il suo ruolo è sempre quello di configurare `log4j` per poi avviare l'esecuzione del task, creando un oggetto che sia istanza della classe dell'algoritmo, e poi chiamando, in ordine, i metodi precedentemente elencati. Il codice di `init` serve per configurare la `SparkSession` e riportare i parametri passati all'applicazione al momento del lancio del task. Il metodo `run` è il corpo centrale di ogni algoritmo: prevede una prima sezione in cui viene caricato il dataset e i dati sono preparati all'elaborazione successiva (solitamente si opera sullo schema per rendere uniforme ogni input), poi c'è una seconda sezione in cui viene effettuato il fit del modello oppure se ne utilizza uno già allenato per fare previsioni. Con `postProcessing` si salvano i risultati, quindi nel caso di task in cui viene allenato un modello, vengono salvati i parametri dello stesso, mentre nei task predittivi vengono scritti in memoria gli output in formato `parquet`. La modalità con cui viene salvato il modello, ovvero con le API di Spark, fa sì che per utilizzare il modello per fare predizioni serva usare nuovamente le API della medesima libreria utilizzata per l'allenamento e il salvataggio.

Il formato Parquet. Apache Parquet è un formato di file binario per l'archiviazione efficiente di dati strutturati. Quando si parla di "formato di file binario", si fa riferimento a un tipo di file che memorizza i dati in un formato non testuale, ottimizzato per le operazioni di elaborazione dei dati. Nasce per affrontare il problema delle query distribuite, dopo la ricerca [23] del 2010, in cui veniva illustrata un'architettura per query distribuite e un formato di archiviazione utile a rendere efficienti le operazioni. In questo contesto furono sviluppate e poi rese *open source* tecnologie come *Apache Impala*, *Presto* e *Apache Parquet*. La caratteristica peculiare di Apache Parquet consiste nel modo in cui i dati vengono organizzati e compressi all'interno dei file. I dati sono memorizzati per colonna anziché per riga, consentendo una compressione più efficiente e una migliore gestione. Infatti, poiché i valori della stessa colonna sono spesso dello stesso tipo, la compressione risulta particolarmente efficiente. Supporta diversi codec di compressione, come Snappy, GZIP, LZO e BROTLI, che consentono di ridurre le dimensioni dei dati memorizzati su disco. Questo porta a una maggiore efficienza complessiva del sistema, riducendo le risorse

di I/O e Central Processing Unit (CPU) necessarie per l'elaborazione dei dati. È stato progettato sin dall'inizio per operare con strutture dati complesse nidificate, dunque utilizza l'algoritmo di frammentazione e assemblaggio dei record descritto in [23]. In questo algoritmo, le strutture dati complesse vengono divise in parti più gestibili, mantenendo comunque la relazione gerarchica tra di esse. Successivamente, nel processo di assemblaggio, i dati frammentati vengono ricomposti per formare nuovamente le strutture dati complesse originali. Grazie al suo modello di archiviazione e alla capacità di supportare la compressione dei dati, Apache Parquet offre elevate prestazioni e una gestione ottimizzata dei dati, rendendolo una scelta popolare per applicazioni come i modelli ML che richiedono un rapido accesso ai dati.

Maven. I progetti Java sono stati gestiti con Apache Maven, uno strumento per l'amministrazione di progetti software, quindi per risolvere le dipendenze e aggiornare la versione di Spark, l'attività svolta è stata quella di scrivere un nuovo file Project Object Model (POM) per ogni algoritmo. Tale file è in formato eXtensible Markup Language (XML) e comprende sia i dettagli di configurazione del progetto, sia le informazioni necessarie a Maven per realizzare l'archivio contenente l'applicazione Java. Il file POM contiene delle sezioni che sono dedicate alla struttura del progetto, ovvero in cui sono indicate le relazioni fra le diverse classi; una sezione dedicata alle proprietà del progetto (strettamente dipendenti dal linguaggio e dalle tecnologie utilizzate); una sezione dedicata alla fase di costruzione del pacchetto dell'applicazione Java, dove possono essere configurati vari plugin; un'ultima sezione in cui sono indicate tutte le dipendenze del progetto. All'interno di un singolo progetto possono esserci diversi file POM, ognuno che gestisce una parte del codice, a diversi livelli di dettaglio: un file più generale può indicare soltanto l'identificativo del gruppo (package) di cui fa parte il progetto, l'identificativo dell'artefatto e i diversi moduli che lo compongono. I file dei singoli moduli possono specificare i plugin necessari alla costruzione del JAR, così come le dipendenze. Alcuni algoritmi prevedono la presenza di un task Spark dedicato soltanto all'allenamento del modello e un task diverso per effettuare predizioni utilizzando il modello allenato; in questo caso è stato organizzato un unico progetto in cui i due task sono moduli di uno stesso artefatto generale. Un esempio di file POM generale è riportato nell'appendice A, mentre un esempio di un file di un singolo modulo, contenente sia i plugin sia le dipendenze, è riportato nell'appendice B. Il funzionamento di Maven si basa sull'utilizzo dei plugin, ovvero degli strumenti che combinano più azioni in sequenza per rendere semplice la compilazione, il testing, il deployment e altre attività comuni durante lo sviluppo software. I plugin vengono configurati all'interno del file POM. Nell'ambito del lavoro di questa tesi, sono

stati configurati, per ciascun task, Apache Maven Assembly Plugin [5] e Log4j Transform Maven Plugin [22]. Il primo consente di costruire un archivio che contiene anche le dipendenze, i moduli, la documentazione, mentre il secondo serve per ottimizzare le chiamate di logging alle API di Log4j, infatti inserisce le chiamate di logging in posizioni statiche, in modo tale da non dover calcolare a runtime la posizione.

Uber jar. Gli applicativi Java sono solitamente "confezionati" in file archivio, chiamati JAR (da Java Archive), che comprendono sia i file di diverse classi Java, sia i relativi metadati e tutte le risorse che utilizzano. Queste risorse generiche possono essere sia file di testo o multimediali, sia altri file JAR. I JAR sono un aspetto fondamentale del linguaggio Java, sia per quanto riguarda la facilità di distribuzione sia per la sicurezza, infatti è possibile inserire in un unico file, detto *uber jar* o *fat jar*, tutti i file delle classi sviluppate per l'applicativo, uniti a tutte le classi che costituiscono le dipendenze, rendendo l'archivio un file completamente autonomo, ossia eseguibile senza necessità di importare nessuna classe esterna.

Aggiornamento della versione di Spark. In particolare, l'attività di aggiornamento e riorganizzazione del codice si è svolta in più fasi in cui sono state prima risolte le dipendenze delle classi relative ai singoli algoritmi, svolgendo un test con un dataset di esempio per ciascun task e lanciando l'esecuzione direttamente dall'interno dell'ambiente di sviluppo. In seguito è stata aggiornata la versione di Spark, e sono state svolte minime modifiche al codice Java delle classi per risolvere alcuni problemi di compatibilità relativi all'aggiornamento, solitamente dovuti al fatto che nella nuova versione di Spark alcune classi sono state spostate in librerie differenti rispetto alla versione 2.2.0. Per l'aggiornamento della classe "VectorDisassembler", dopo aver effettuato la *fork* del codice, si è aggiornato il file POM, poi si è costruito il nuovo file JAR che è stato in seguito importato da tutti i task che lo richiedevano.

Il plugin Maven Assembly è stato configurato in modo tale che venisse incluso il *Classpath* (`<addClasspath>true</addClasspath>`), è stata specificata la classe principale da eseguire una volta avviato l'archivio finale (`<mainClass>it.unimi.evotion.tasks.kmeans_model.KmeansModel</mainClass>`), è stato inserito il tipo di descrittore da utilizzare per nominare l'assembly JAR (`<descriptorRef>jar-with-dependencies</descriptorRef>`) e poi si è specificata la fase del ciclo di vita di Maven in cui eseguire l'istanza di esecuzione (`<phase>package</phase>`). Una volta aggiornati i file di configurazione di tutti i progetti relativi ai singoli algoritmi, sono stati costruiti i file *uber jar*.

Capitolo 5

Esecuzione in ambiente gestito con Kubernetes

1 Esecuzione su cluster locale

Al fine di rendere eseguibili i task Spark in un ambiente distribuito gestito con Kubernetes, è stato fatto un primo test di esecuzione in Container Docker dei singoli *fat jar*, in modo tale da individuare un'immagine di partenza che soddisfacesse le necessità delle singole classi. Successivamente si è anche effettuata una prova di esecuzione su un cluster Kubernetes locale, ovvero eseguito su una sola macchina. In questa fase, sia il file archivio del task sia il dataset di lavoro sono stati aggiunti all'immagine Docker utilizzata per la costruzione dei Pod Kubernetes, tuttavia nella configurazione finale, come verrà illustrato in seguito, il file eseguibile e i dati non sono inseriti nell'immagine, ma vengono letti da un sistema di storage distribuito. Per costruire un'immagine Docker da usare in Kubernetes, è stato usato lo strumento `docker-image-tool.sh`, messo a disposizione dalla distribuzione Spark. L'immagine di partenza è presente su Docker Hub con tag `eclipse-temurin:8-jre`, ed è sviluppata nell'ambito del progetto Eclipse Temurin. Si tratta di una distribuzione linux Ubuntu 22.04 su cui è stato installato OpenJDK.

Configurazione dell'ambiente di lavoro. Il cluster locale è stato configurato su un dispositivo Apple MacBook Pro con processore Apple M1 Pro e 16 GB di memoria unificata, impostando l'applicativo Docker Desktop (utilizzato anche per la gestione del cluster Kubernetes) in modo tale che potesse usare fino a 8 core di CPU e 13 GB di memoria. Da Docker Desktop è stato attivato Kubernetes, in modo tale che venisse configurato un cluster con un singolo nodo che contiene i Pod dedicati a: API Server, Controller Manager, Scheduler,

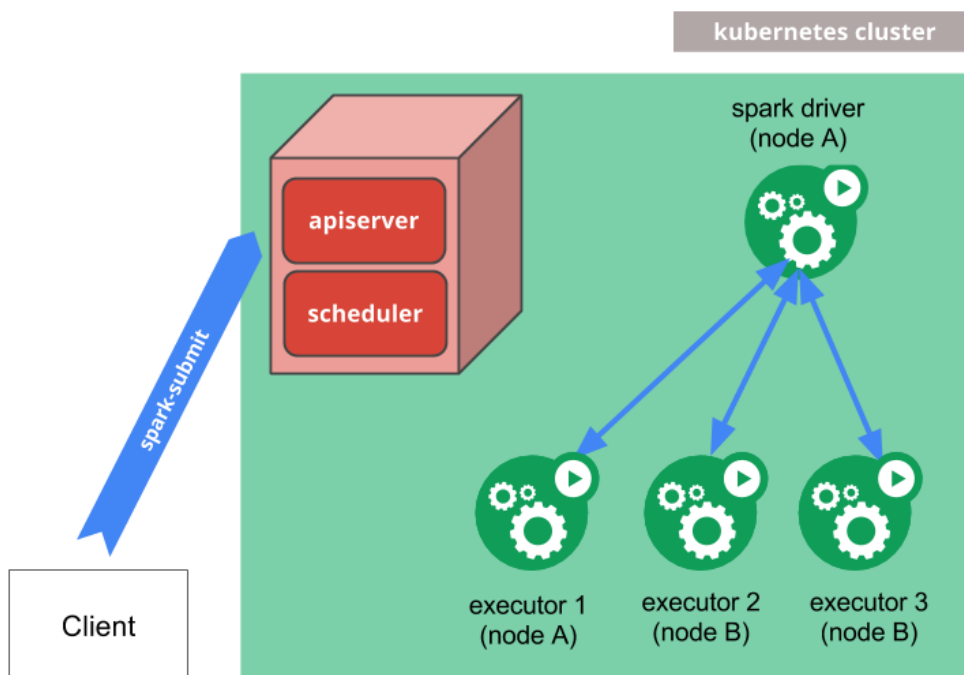


Figura 5.1: Funzionamento di spark-submit

Fonte: <https://spark.apache.org/docs/3.5.0/running-on-kubernetes.html>

storage etcd, coredns, kube-proxy (si faccia riferimento alla sezione 4 per la spiegazione delle funzioni dei singoli Pod). Una volta configurato il cluster, si è utilizzato lo script `spark-submit` per avviare l'esecuzione dei task; con questa procedura viene impartito il comando all'API server che si occupa di creare, all'interno dell'unico nodo del cluster, un nuovo Pod che funge da Spark driver. A sua volta, il driver crea gli esecutori e si connette a tali Pod, per poi eseguire il codice dell'applicazione.

Implementazione. Per rendere più agevole l'attività di recupero degli algoritmi, si è scelto di configurare le proprietà di Spark non attraverso le API di Java, ma con i parametri dello strumento `spark-submit`. In questo modo non è necessario modificare il codice e in più se ne favorisce il riuso, perché al variare dell'infrastruttura di deployment non serve apportare modifiche, ma basta cambiare i parametri dello strumento di lancio. Questo aspetto, come più volte ripreso in questo elaborato, è fondamentale per l'attività svolta in questa tesi. Di seguito si riportano le impostazioni utilizzate per il lancio sul cluster locale:

1. `--master` serve per indicare l'URL dello Spark master, in particolare l'indirizzo dell'API server Kubernetes;

2. `--deploy-mode` definisce dove istanziare il nodo driver, in modalità `client` è avviato localmente e quindi si interfaccia ai nodi del cluster come nodo esterno, mentre in modalità `cluster` viene avviato direttamente sui nodi worker;
3. con `--class` si specifica la classe da eseguire all'interno dell'archivio JAR;
4. il termine indicato in `--name` è quello utilizzato da Kubernetes per nominare le risorse create per l'esecuzione dei task, come i driver e gli executors;
5. attraverso l'utilizzo di `--conf` si possono impostare diverse proprietà per la configurazione di Spark, in particolare è utile indicare il namespace in cui avviare i Pod per eseguire il task (con `spark.kubernetes.namespace`) e l'immagine con cui avviare i Container (con `spark.kubernetes.container.image`), inoltre nel caso in cui sia necessaria l'autenticazione per avere il permesso di esecuzione all'interno del cluster, si indica il nome del Service Account in Kubernetes con `spark.kubernetes.authenticate.driver.serviceAccountName`.

Oltre ai parametri di configurazione, lo strumento richiede che venga indicato il JAR dell'applicazione da eseguire ed eventuali argomenti. Nel caso dei task Spark coinvolti nel lavoro, è stato indicato il percorso interno all'immagine in cui è stato inserito l'archivio Java, il percorso del dataset e gli argomenti specifici per ciascun modello.

2 Esecuzione su cluster AWS EKS

Configurazione dell'ambiente di lavoro. Per avvicinarsi alla configurazione di deployment finale, si è configurato un cluster Kubernetes utilizzando il servizio di cloud computing EKS offerto da AWS. Si è scelto di utilizzare il servizio interamente in cloud, quindi sia il Kubernetes Control Plane, sia il data plane sono eseguiti in data center forniti da AWS. L'esecuzione delle applicazioni sul cluster avviene attraverso EMR, un servizio che utilizza i parametri della *job definition* per richiedere a EKS le risorse (in termini di Pod e Container) necessarie.

Gestione degli accessi. Per poter accedere al cluster dalla CLI, si è creata una chiave di accesso per l'utente *admin* dall'interfaccia web di AWS, poi si è installato sul dispositivo locale lo strumento AWS CLI e si è configurato con la chiave creata. Al fine di gestire il cluster EKS, si è creato un IAM User

Group denominato EKSAdministrators con policy di accesso Full Access per le risorse EKS. L'utente *admin* è stato inserito nel gruppo appena creato, in questo modo è autorizzato a creare, modificare, eliminare cluster EKS, gestire i nodi del cluster, configurare le autorizzazioni e le policy di accesso, nonché eseguire altre attività amministrative all'interno dell'ambiente EKS. Dal momento che le risorse del cluster vengono amministrate con lo strumento kubectl, lo si deve configurare per accedere a EKS. La configurazione è possibile solamente una volta creato e avviato il cluster remoto, dunque è descritta in seguito.

Creazione del cluster. Sfruttando lo strumento eksctl si è creato un cluster di istanze di tipo Elastic Compute Cloud (EC2) *m5.xlarge* —indicate per computazione "general batch processing"— dotate di 4 CPU virtuali e 16 GiB di memoria, ed è stato esposto l'accesso remoto tramite ssh.

```
eksctl create cluster \
--name test-cluster \
--region us-east-1 \
--with-oidc \
--ssh-access \
--ssh-public-key key-pair-test-cluster \
--instance-types=m5.xlarge \
--managed
```

Si è poi aggiornato il file di configurazione di kubectl per accedere al cluster creato. La modifica della configurazione avviene in maniera automatica utilizzando il comando `aws eks update-kubeconfig`, in particolare vengono aggiunti il contesto

```
- context:
  cluster: test-cluster.us-east-1.eksctl.io
  user: admin@test-cluster.us-east-1.eksctl.io
  name: admin@test-cluster.us-east-1.eksctl.io
```

e viene definito l'utente `admin@test-cluster.us-east-1.eksctl.io` in modo tale che sia autenticato attraverso `aws-iam-authenticator`.

Configurazione del servizio EMR. All'interno del cluster si è creato un namespace dedicato all'esecuzione dei task (denominato `task-namespace`), poi una mappatura (*iamidentitymapping*) fra uno specifico utente Kubernetes e il ruolo dedicato all'amministrazione dei Container. In questo modo EMR ha accesso al namespace in cui devono essere creati i Pod per l'esecuzione dei task. Tutte le operazioni di configurazione sono gestite in automatico da eksctl, attraverso il comando `create iamidentitymapping`.

Abilitazione di IAM Roles for Service Accounts. Per rendere possibile associare direttamente ruoli IAM ai servizi all'interno del cluster, consentendo loro di accedere alle risorse AWS senza la necessità di utilizzare credenziali statiche all'interno dei Container, si deve creare un Identity Provider per il cluster. La creazione del provider, compatibile con protocollo di autenticazione OpenID Connect, avviene attraverso il comando `eksctl utils associate-iam-oidc-provider --cluster test-cluster --approve`. In questo modo i Pod all'interno del cluster possono assumere ruoli IAM specifici per accedere alle risorse AWS in modo sicuro e controllato.

Configurazione del *job execution role*. È stato creato un ruolo IAM per l'esecuzione dei task e vi è stata associata una policy che consentisse l'accesso completo ad uno specifico bucket Amazon Simple Storage Service (S3) e al servizio di monitoraggio Amazon CloudWatch. Si è poi aggiornata la trust policy del cluster in modo tale che l'utente *admin* potesse assumere il ruolo appena creato.

```
aws emr-containers update-role-trust-policy \
--cluster-name test-cluster \
--namespace task-namespace \
--role-name EMRContainers-JobExecutionRole
```

Un ulteriore permesso necessario per l'esecuzione di task sul cluster EKS è quello di amministrazione dei cluster virtuali e lancio di "job" software. Si è dunque creata una policy (AmazonEMRONEKSPolicy) contenente tali permessi e la si è associata all'utente *admin* con il comando ([ID] sostituisce il reale ID del cluster)

```
aws iam attach-user-policy \
--user-name admin \
--policy-arn arn:aws:iam::[ID]:policy/AmazonEMRONEKSPolicy
```

Si faccia riferimento all'appendice C per il contenuto delle policy citate.

Creazione del cluster virtuale. Infine è stato creato un cluster virtuale per l'esecuzione vera e propria dei task, indicando come provider dei Container il cluster fisico appena configurato, in particolare il namespace creato precedentemente.

```
aws emr-containers create-virtual-cluster \
--name tasks-virtual \
--container-provider '{
  "id": "test-cluster",
  "type": "EKS",
  "info": {
    "eksInfo": {
```

```

        "namespace": "task-namespace"
    }
  },
}
```

Implementazione. Con un'infrastruttura così configurata, è possibile eseguire i task utilizzando direttamente lo strumento `spark-submit`, come precedentemente illustrato, ma eseguendolo da un nodo appartenente al cluster. In alternativa si può utilizzare EKS StartJobRun, uno strumento pensato appositamente per l'esecuzione di Spark task in un cluster EKS, attraverso il quale viene configurato indirettamente `spark-submit`.

Esecuzione con `spark-submit`. Prima di eseguire l'applicazione, va creato un Service Account Kubernetes che si lega al Cluster Role con il permesso di modifica del cluster, creando un Cluster Role Binding:

```
kubectl create clusterrolebinding spark-role --clusterrole=edit
--serviceaccount=task-namespace:default
```

A questo punto, si crea un Container utilizzando l'immagine Docker fornita da AWS appositamente per l'esecuzione di task Spark.

```
kubectl run -it testcontainer --image=755674844232.dkr.ecr.us-
east-1.amazonaws.com/spark/emr-7.0.0:latest --command -n task-
namespace /bin/bash
```

Dal processo `bash` in esecuzione nel Container, si esegue il comando per l'avvio del task (in questo caso il semplice esempio già presente nell'immagine Spark)

```
$SPARK_HOME/bin/spark-submit \
--class org.apache.spark.examples.JavaSparkPi \
--master $MASTER_URL \
--conf spark.kubernetes.container.image=755674844232.dkr.ecr.us-
east-1.amazonaws.com/spark/emr-7.0.0:latest \
--conf spark.kubernetes.authenticate.driver.serviceAccountName=
default \
--deploy-mode cluster \
--conf spark.kubernetes.namespace=task-namespace \
local:///usr/lib/spark/examples/jars/spark-examples.jar 20
```

dove `MASTER_URL` è la variabile d'ambiente in cui è memorizzato l'indirizzo del Control Plane del cluster EKS preceduto da `k8s://`. È da notare che si utilizza il Service Account precedentemente creato per l'autenticazione.

Esecuzione con `StartJobRun`. Nell'appendice D si può vedere l'esempio del file JSON per la configurazione dello strumento. La configurazione mostrata è per l'esecuzione del task che implementa l'algoritmo `kmeans`: a differenza

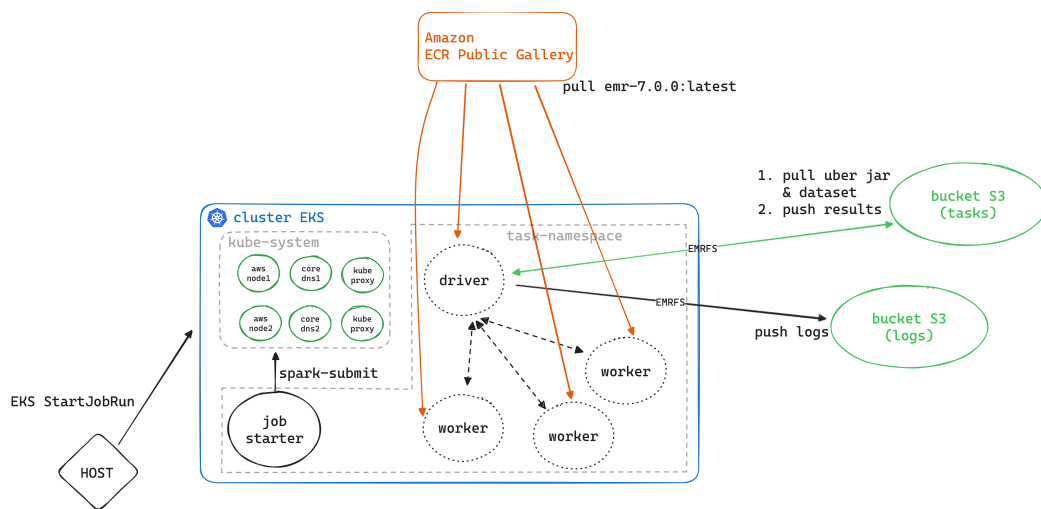


Figura 5.2 : Schema riassuntivo dell'infrastruttura configurata con Amazon EKS

dell'esecuzione su cluster locale, l'archivio JAR e il dataset passati al tool di esecuzione, così come il percorso in cui scrivere i risultati dell'algoritmo, non sono locali all'immagine eseguita dal Container nel cluster, ma sono percorsi in un bucket Amazon Simple Storage Service (S3). La comunicazione fra lo Spark driver e il bucket S3 avviene attraverso EMR File System (EMRFS), un'implementazione di Hadoop Distributed File System (HDFS) usata per leggere e scrivere file dai cluster EMR a S3 [15]. In aggiunta ai parametri di spark-submit, nel file di configurazione vanno indicati l'identificativo del cluster virtuale, l'Amazon Resource Name (ARN) del Job Execution Role, la versione di EMR da utilizzare, le risorse hardware da assegnare allo Spark driver e la configurazione di CloudWatch.

Capitolo 6

Studio e implementazione di un algoritmo di test in Tensorflow

Parallelamente all'attività di recupero degli algoritmi Spark, si è voluta studiare la possibilità di utilizzare lo stesso ambiente distribuito con Kubernetes per eseguire task DL. Come illustrato nella sezione 1, la libreria Tensorflow è estremamente utilizzata in molti ambiti del machine learning, grazie alla sua semplicità d'uso e alla possibilità di ottimizzare l'esecuzione su hardware specifico. Si è studiata, dunque, l'implementazione di nuovi algoritmi sfruttando la libreria in questione, superando l'approccio con Java, pur mantenendo la possibilità di esecuzione sul cluster precedentemente configurato. Per farlo, è stato utilizzato un popolare modello DL che implementa una rete neurale convoluzionale per riuscire a classificare le immagini di cifre scritte a mano, di fatto riconoscendo la cifra presente nell'immagine. Il dataset [21] utilizzato per la costruzione del modello è composto da 60000 esempi dedicati al training e 10000 dedicati al testing; è presente direttamente in Tensorflow (`tf.keras.datasets.mnist`).

1 Analisi

Tensorflow ha la sua API —`tf.distribute`— che permette la distribuzione sia della computazione da parte di modelli predittivi già allenati, sia del training; è pensata per funzionare sia con il codice che utilizza API fornite da librerie di alto livello, come `Model.fit` di Keras, sia con programmi che usano Tensorflow più a basso livello, come quelli con *training step* personalizzati [13]. La grande automazione presente nei componenti della libreria Tensorflow consente di distribuire la computazione del codice applicando delle modifiche minime: occorre definire una "strategia" di distribuzione, utilizzare metodi specifici per configurare i dispositivi coinvolti in modo da comunicargli il ruolo che devo-

no assumere nella computazione, creare il modello all'interno dello *scope* della strategia appena definita. Di seguito si discute nel dettaglio ciascun passaggio.

2 Configurazione dell'ambiente di lavoro

Definizione della strategia. Le strategie si differenziano sia per la tipologia di piattaforma hardware che utilizzano, sia per la modalità di parallelizzazione del training che sfruttano. Quest'ultima può essere sincrona, quindi con tutti i *workers* che lavorano contemporaneamente su diverse sezioni del dataset di input e poi effettuano una sincronizzazione ad ogni step, oppure asincrona, ovvero con un lavoro indipendente e aggiornamento asincrono delle variabili. Esistono 7 differenti strategie di distribuzione del calcolo:

1. per distribuire il training in maniera sincrona su più Graphics Processing Unit (GPU) su uno stesso dispositivo, viene utilizzata `MirroredStrategy` con cui viene creata una replica per ogni variabile del modello su ciascuna GPU. In seguito i valori sono tenuti sincronizzati e vengono aggiornati utilizzando efficienti algoritmi all-reduce che solitamente usano NVIDIA Collective Communications Library (NCCL);
2. `TPUStrategy` viene usata per effettuare il training distribuito su un cluster di Tensor Processing Unit (TPU);
3. la strategia adottata in questo lavoro, `MultiWorkerMirroredStrategy`, serve per la distribuzione sincrona su diversi dispositivi. Offre la possibilità di avere più GPU su ciascun dispositivo e crea una copia di ogni variabile del modello su ciascun worker. La comunicazione fra i dispositivi può avvenire con Remote Procedure Call (RPC) (compatibile CPU e GPU) o NCCL (solo per GPU);
4. con `ParameterServerStrategy` si implementa un cluster per il training asincrono, composto da parameter server e da worker. I primi creano e conservano le variabili del modello, mentre i secondi leggono le variabili dai server per poi aggiornarle e riscriverle sui server, senza mai sincronizzarsi fra loro;
5. un'altra strategia utilizzabile su un solo dispositivo è `CentralStorageStrategy`, la quale prevede che le variabili siano soltanto sulla CPU, mentre le operazioni di aggiornamento sono svolte distribuite sulle GPU;
6. `OneDeviceStrategy` serve per memorizzare tutte le variabili ed eseguire qualsiasi funzione su uno specifico dispositivo, come una singola GPU;

7. la strategia di default, ovvero quella utilizzata nel caso in cui non venisse definita una delle altre, non distribuisce il carico computazionale. È utilizzata soltanto per scrivere librerie o programmi che possano gestire ogni tipo di distribuzione e funzionino anche senza distribuire la computazione.

Configurazione. Una volta definita la strategia, per parallelizzare la computazione su un cluster bisogna configurare i dispositivi coinvolti in modo tale che conoscano il ruolo che devono assumere nel training o nell'utilizzo del modello predittivo. Nel caso di `TPUStrategy` si utilizzano direttamente le API `Tensorflow` per indicare l'indirizzo del TPU cluster resolver, connettersi e inizializzare il sistema, poi la distribuzione è gestita in maniera automatica dalla libreria. Con strategie che coinvolgono realmente più worker, come `MultiWorkerMirroredStrategy` e `ParameterServerStrategy`, si ha un controllo preciso su tutti i dispositivi del cluster. Nella variabile d'ambiente `TF_CONFIG`, scritta con formato JSON, si configura la sezione `cluster`: un dizionario contenente gli indirizzi IP e i numeri di porta di tutti i nodi del cluster, distinguendo fra worker ed eventualmente parameter server (`ps`) (è un'informazione uguale per tutti i nodi). Inoltre, va definita la sezione `task` che specifica il tipo e l'indice di un singolo nodo. Il dispositivo con indice pari a 0 per prassi è il nodo chief che, oltre a svolgere i compiti di un normale worker, si occupa anche di salvare i checkpoint e scrivere i *summary files* per TensorBoard. In aggiunta alla configurazione dei singoli nodi, è possibile definire la modalità di comunicazione fra dispositivi, utilizzando le API di `Tensorflow` e scegliendo fra `RPC` (`CommunicationImplementation.RING`) e `NCCL` (`CommunicationImplementation.NCCL`).

L'unica modifica al codice necessaria per distribuire il training, una volta definita la strategia, è quella di effettuare le chiamate alle API per costruire e compilare il modello all'interno dello scope della strategia.

3 Implementazione

La rete CNN, implementata con l'API `Keras`, è composta da 6 layer. L'input deve avere forma 28x28, quindi le immagini sottoposte al modello devono essere quadrate e avere lato di 28 pixel. Un secondo layer (`Reshape`) si occupa di trasformare le immagini in `Tensors` tridimensionali. Segue un layer convoluzionale (`Conv2D`) che utilizza 32 filtri, quindi l'output ha dimensione 32, e un kernel di dimensione 3, inoltre usa "relu" (`REctified Linear Unit`) come funzione di attivazione; questo è il layer in cui viene effettivamente estratta l'informazione dall'immagine. Con il quarto layer (`Flatten`) si riducono i `Tensors` tridimensio-

nali a un vettore unidimensionale, in modo tale da poter sfruttare gli ultimi due layer che sono densamente connessi. Il penultimo layer serve a introdurre non linearità nel modello e così consentirgli di apprendere anche pattern complessi, dunque ha funzione di attivazione "relu" e output di dimensione 128. L'ultimo layer si occupa di restituire la classificazione dell'immagine in ingresso, quindi ha una funzione di attivazione lineare (di default) e 10 possibili output.

3.1 Deployment su cluster Kubernetes

Per eseguire un task in maniera distribuita, è necessario implementare un cluster Kubernetes composto da tanti Pod quanti sono i nodi worker su cui si desidera distribuire il carico computazionale. Ogni singolo Pod deve avere risorse sufficienti all'allenamento del modello, quindi se si desidera utilizzare la GPU, è importante configurarla al momento della creazione del Pod. Oltre alla gestione dell'hardware, serve impostare ogni worker in modo tale che abbia accesso al codice python per il training e configurare le variabili d'ambiente per la gestione della distribuzione. Per fare in modo che ogni Pod del cluster possa raggiungere gli altri (operazione necessaria per la sincronizzazione), va creato un Kubernetes Service attraverso il quale si espone una porta del Container verso la rete interna al cluster.

L'esecuzione svolta sul cluster locale ha coinvolto tre worker, dunque la variabile TF_CONFIG è stata configurata come segue (il valore di \$index varia fra 0 e 2).

```
TF_CONFIG='{
  "cluster": {
    "worker": [
      "worker-0:2222",
      "worker-1:2222",
      "worker-2:2222"
    ]
  },
  "task": {
    "type": "worker",
    "index": '$index'
  }
}'
```

Con i file di configurazione in formato YAML si stabiliscono le risorse da allocare per ciascun Pod, oltre alla configurazione hardware e software dei vari nodi. Per ogni Pod è stato impostato un Container con nome "tensorflow", con 1 Gi di memoria e 1 CPU, si è inizializzata la variabile TF_CONFIG e si è montato un volume per contenere lo script. Nella sezione "command" è stato definito il comando per avviare il codice python alla creazione del Pod. L'immagine

utilizzata per la creazione del Container è stata costruita a partire da quella ufficiale python 3.9, in cui sono stati installati i moduli tensorflow e numpy. Questa procedura si è resa necessaria per eseguire il modello in un cluster locale su architettura arm64, nel momento in cui si esegue su amd64 (x86-64), si può semplicemente utilizzare l'immagine ufficiale Tensorflow. Per ogni worker è stato scritto anche un file YAML per la creazione del Service in modo tale da esporre, con protocollo TCP, la porta 2222. Questa è raggiungibile solamente all'interno del cluster perché la comunicazione che avviene per il training distribuito è soltanto fra i nodi interni al cluster. Il codice è stato caricato configurando il volume montato in ciascun Pod per leggere i dati da un Kubernetes ConfigMap, all'interno del quale è stata definita una sezione `main.py` al campo "data" che contiene il codice python. La scelta dell'utilizzo di ConfigMap si deve al fatto che in questo modo è molto semplice rendere disponibile uno script sviluppato localmente a tutti i Pod, senza dover, per esempio, inserire il codice all'interno dell'immagine Docker. Un'altra opzione adatta a questo contesto è l'uso di una repository Git come punto di partenza del volume montato nei Pod, infatti anche in questo caso la distribuzione del codice sarebbe immediata, ma i nodi andrebbero riavviati nel momento in cui si dovesse modificare il codice. In produzione, la situazione ideale dovrebbe prevedere la creazione di un'immagine Docker che contenga l'applicazione, in questo modo i Pod potrebbero semplicemente scaricare tale immagine ed eseguire il codice. È bene notare che in questo modo la procedura di configurazione del cluster Kubernetes sarebbe completamente separata dal modello che si vuole allenare. Sia il Service sia i Pod vengono creati attraverso lo strumento `kubectl`, nello specifico si passa il nome del file in formato YAML al comando `kubectl apply -f`. Una volta create le risorse, con `kubectl logs -f worker0` è possibile seguire l'avanzamento del training del modello sul worker chief. Con un comando analogo è possibile visualizzare anche i log degli altri worker.

Deployment su un cluster Spark esistente. Qualora si volesse distribuire il training su un cluster Spark già esistente, all'interno dell'ecosistema Tensorflow è stato sviluppato un apposito componente che consente la distribuzione in maniera immediata. La funzione `MirroredStrategyRunner` all'interno del pacchetto `spark_tensorflow_distributor` può essere utilizzata in un modo del tutto simile a quello descritto nei precedenti paragrafi, infatti è costruita sulla base di `MultiWorkerMirroredStrategy`. L'utilizzo classico prevede di incapsulare il codice da eseguire su un singolo nodo all'interno di una funzione, poi si passa tale funzione a `MirroredStrategyRunner.run()`. Il runner può essere configurato con i seguenti parametri: il numero totale di GPU o CPU utilizzabili da Spark (`num_slots`), se deve essere eseguito in

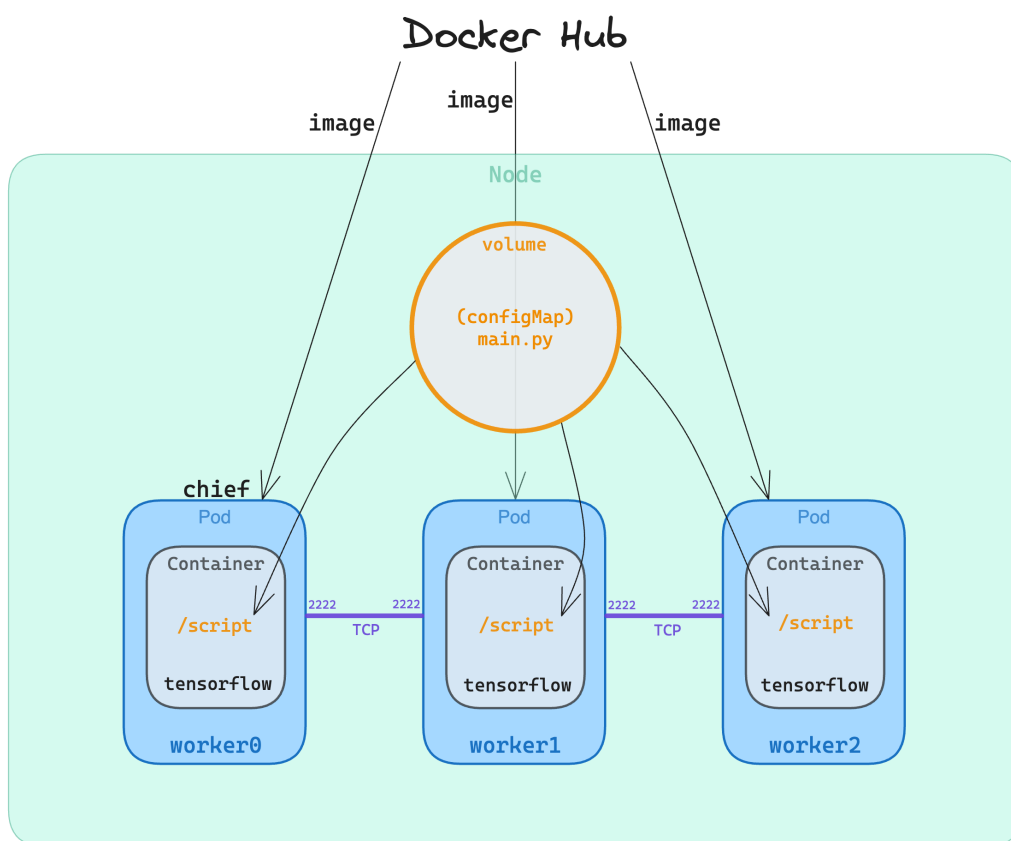


Figura 6.1 : Schema di funzionamento della distribuzione di tensorflow su cluster Kubernetes

locale sullo Spark driver o distribuito sui worker (`local_mode`), se deve utilizzare la GPU (`use_gpu`), il nome della risorsa che Spark usa per lo scheduling della GPU (`gpu_name`), se il codice fornito per il singolo worker implementa una strategia personalizzata —ovvero una di quelle elencate precedentemente— o meno (`use_custom_strategy`). Un aspetto fondamentale della modalità `use_custom_strategy=False` è che `MirroredStrategyRunner` incapsula il codice in una strategia `MultiWorkerMirroredStrategy`. Ciò consente di distribuire del codice che non prevede nessuna modalità di distribuzione, senza intervenire direttamente sul codice stesso.

Tensorflow Serving. In modo simile a quanto avviene per i modelli Spark, con Tensorflow è possibile utilizzare il sistema *Serving* così da sfruttare un modello già allenato per effettuare predizioni. Una volta allenato un modello, si salva utilizzando l'API di Keras `save_model`. Con questa operazione viene creata una directory che contiene il modello, descritto come un'insieme di dataflow graph, e dei file contenenti le variabili del modello. Una volta salvato, si configura un server che espone una porta sulla quale può ricevere dati per fare la predizione, utilizzando il modello. In particolare, si utilizza l'immagine Docker `tensorflow/serving` e si monta all'interno del Container la directory contenente il modello. Le richieste al server vengono effettuate tramite RPC asincrone. Il fatto che il meccanismo sia pensato per l'esecuzione in Container, rende molto facile la costruzione di un ambiente Kubernetes in cui si configurano diversi Pod del cluster dedicati a fare previsioni con modelli pre-allenati, come riportato nella documentazione ufficiale.

Capitolo 7

Deployment sull'infrastruttura MUSA e testing

1 Architettura MUSA

Il cluster Kubernetes sviluppato nell'ambito del progetto MUSA è composto da 7 nodi, di cui 3 master (sui quali è in esecuzione il control plane e conservato etcd) e 4 worker. Tutti hanno CPU con architettura amd64 ed eseguono la distribuzione linux Ubuntu 22.04.3 LTS. Il control plane è amministrato con lo strumento Rancher, con il quale è gestita anche l'autenticazione RBAC. Ai fini di questo lavoro sono state utilizzate le risorse nel namespace "musa" all'interno del quale è definito il Service Account "spark", necessario per avere i permessi di esecuzione dei task (maggiori dettagli in seguito).

Nel namespace "musa" sono sempre in esecuzione un Pod dedicato allo spark-history server, uno ad Apache Livy ed uno per Apache Kyuubi. Il primo mantiene i log di tutte le applicazioni Spark eseguite nel namespace e le metriche per valutare l'esecuzione. Livy è un sistema che fornisce un'interfaccia REST per interagire con il cluster da remoto. Questo permette di inviare job Spark, monitorarne lo stato e ottenere i risultati attraverso chiamate API HTTP. Kyuubi fornisce un server JDBC per Spark, rendendo possibile eseguire query Structured Query Language (SQL) sul cluster utilizzando connessioni standard JDBC. Ogni Pod espone le porte di cui necessita per erogare il rispettivo servizio con un Kubernetes Service di tipo ClusterIP. I Service sono configurati con indirizzi IP scelti e statici.



Figura 7.1 : Meccanismo per l'implementazione dell'autenticazione basata sui ruoli all'interno del cluster Kubernetes MUSA

2 Processo di deployment

In maniera analoga a quanto fatto sul cluster locale e su AWS EKS, l'obiettivo è quello di utilizzare lo strumento `spark-submit` per eseguire i task Spark sul cluster Kubernetes implementato nell'architettura MUSA. Come precedentemente illustrato, per interagire con Kubernetes si utilizza `kubectl`. Dal momento che, come illustrato nella sezione precedente, l'autenticazione è gestita attraverso Rancher, il primo passaggio effettuato è stato quello di configurare `kubectl` in modo tale che avesse le credenziali per accedere al cluster. All'interno del file di configurazione, fornito direttamente da Rancher, sono riportati l'URL del servizio Rancher nel cluster, i dati del certificato dell'autorità di certificazione per autenticare il server (nella sezione "clusters"), e il token per l'autenticazione dell'utente (nella sezione "users"). Una volta configurato `kubectl`, si è avviato un application-level gateway con `kubectl proxy` in modo tale che tutto il traffico in entrata all'indirizzo `localhost` sulla porta 8001 venisse inoltrato al Kubernetes API server remoto. A questo punto si è potuto configurare `spark-submit` per eseguire i task Spark nel cluster: come indirizzo del master si è indicato quello del proxy (`k8s://http://127.0.0.1:8001`), inoltre si è indicato il namespace "musa" e il Service Account "spark". Quest'ultimo serve per consentire ai processi in esecuzione all'interno dei Pod di autenticarsi presso l'API server. In particolare, al Service Account "spark" è assegnato, tramite un `ClusterRoleBinding`, il `ClusterRole` "spark" che garantisce il permesso di fare qualsiasi operazione con le risorse pods, services, secrets, configmaps, persistentvolumeclaims.

Di seguito si analizza l'esecuzione dell'algoritmo Linear Regression utilizzando un'immagine Docker creata a partire da `eclipse-temurin:8-jre` per piatta-

forma linux/amd64. Tale immagine contiene l'uber jar del task e il dataset da utilizzare, i risultati vengono scritti nella cartella RESULT del Pod stesso.

```
spark-submit \
--master k8s://http://127.0.0.1:8001 \
--deploy-mode cluster \
--name lr \
--class it.unimi.evotion.tasks.lr.LReg \
--conf spark.kubernetes.container.image=gabrielestentella/spark:
    lramd\
--conf spark.kubernetes.driver.pod.name=lrsparkdriver \
--conf spark.kubernetes.namespace=musa \
--conf spark.kubernetes.authenticate.driver.serviceAccountName=
    spark \
--verbose \
local:///opt/spark/jars/lr-1.0-jar-with-dependencies.jar /opt/
    spark/dataset/ds_nonum.csv HA_USAGE HA_USAGE_SCALED 10 5 0.8
    /opt/spark/RESULT
```

Events:

(Reason) Message

(Scheduled) Successfully assigned musa/lrsparkdriver to
almavivaw2

(Pulled) Container image "gabrielestentella/spark:lramd"
already present on machine

(Created) Created container spark-kubernetes-driver
(Started) Started container spark-kubernetes-driver

Nota: La sezione "Events" è stata modificata per motivi di impaginazione

Quando viene eseguito il comando, lo scheduler Kubernetes assegna un nodo del cluster (almavivaw2), poi kubelet effettua il pull dell'immagine Docker se non è già presente sulla macchina, infine viene creato e poi avviato il Container dedicato al driver. Quando comincia l'esecuzione del task sul driver, questo si occupa di richiedere che vengano istanziati gli esecutori per distribuire la computazione.

3 Testing e valutazione delle performance in ambiente distribuito

Per dimostrare l'effettiva distribuzione del carico computazionale sul cluster si sono svolte molteplici esecuzioni degli algoritmi ML sia sull'infrastruttura Kubernetes locale, sia su quella sul servizio di cloud computing. Si è poi mi-

surato il tempo di esecuzione ed analizzati sia i file di log sia le informazioni riguardanti lo stato del cluster nel corso dell'esecuzione dei task.

3.1 Spark task

Il flusso di lavoro generale atteso e che si è osservato è il seguente:

- utilizzando lo strumento EKS StartJobRun viene creato un Pod interno al cluster per lanciare il task;
- il nodo dedicato allo Spark master richiede a Kubernetes un numero di esecutori (executors) pari a quello indicato nella fase di lancio del workload, quindi viene creato un numero di Pod pari al numero di executors;
- gli indirizzi IP di ciascun executor sono registrati nel cluster e gli viene assegnato un numero identificativo;
- lo Spark master assegna l'esecuzione di un task a ogni executor, fino a quando non sono finiti tutti i task set di tutti i passi che compongono il workload;
- una volta che tutti i task sono stati completati e i nodi executors hanno riportato il risultato allo Spark driver, il lavoro è concluso, quindi gli executors vengono terminati e il Pod dedicato allo Spark driver entra nello stato "completed" (qualora il workload prevedesse la scrittura dei risultati in specifici file, tale azione viene fatta dal driver).

Dall'analisi dei file di log —di cui si riporta un estratto per quanto riguarda l'esecuzione di un algoritmo sul cluster AWS EKS nell'appendice E— si è trovata conferma della corretta esecuzione. Di seguito si descrive in dettaglio l'evoluzione dello stato del cluster, riportando gli indirizzi IP dei singoli Pod durante l'esecuzione del medesimo task per cui si è riportato l'estratto dei file di log.

- Appena lanciato il comando EKS StartJobRun viene creato il primo Pod che si occupa di lanciare il task vero e proprio (IP 192.168.23.130).
- Viene creato lo Spark driver (IP 192.168.36.97) che richiede a Kubernetes le risorse di cui ha bisogno.
- A questo punto vengono registrati i 4 esecutori, ovvero i nodi worker (IP 192.168.3.216, 192.168.58.9, 192.168.27.77, 192.168.47.240).

- Il driver assegna un task appartenente al primo Stage di lavoro ad ogni worker: "Starting task 0.0 in stage 0.0 (TID 0) (192.168.58.9, executor 2, partition 0, PROCESS_LOCAL, 1007791 bytes)". In questa fase tutti i Pod sono nello stato "Running".
- Quando un worker ha terminato il proprio lavoro, riporta il risultato al driver che gli assegna un nuovo task.
- Nel momento in cui tutti gli Stage sono terminati, viene stoppato lo SparkContext. Tutti i Pod dei nodi worker entrano nello stato "Terminating".
- Appena tutti i nodi worker sono stati fermati, il driver entra nello stato "Completed". Accedendo ai suoi log è possibile vedere l'esecuzione del singolo task.

3.2 Tensorflow task

Appena un Pod viene creato, dal momento che tramite il Kubernetes Service ha una porta esposta verso la rete, può subito connettersi agli altri nodi worker. I primi log presentati sono infatti relativi alla connessione, in particolare sul Pod con "index": 0 (chief) si legge

```

1 I tensorflow/core/distributed_runtime/rpc/grpc_server_lib.cc
  :449] Started server with target: grpc://worker0:2222
2 I tensorflow/tsl/distributed_runtime/coordination/
  coordination_service.cc:535] /job:worker/replica:0/task:0 has
  connected to coordination service. Incarnation:
  2448655666425437712
3 I tensorflow/tsl/distributed_runtime/coordination/
  coordination_service_agent.cc:298] Coordination agent has
  successfully connected.
4 I tensorflow/tsl/distributed_runtime/coordination/
  coordination_service.cc:535] /job:worker/replica:0/task:1 has
  connected to coordination service. Incarnation:
  16697895956226192157
5 I tensorflow/tsl/distributed_runtime/coordination/
  coordination_service.cc:535] /job:worker/replica:0/task:2 has
  connected to coordination service. Incarnation:
  6829625911175341699

```

La prima riga indica che un server gRPC è stato avviato sul nodo worker0 sulla porta 2222. Questo server verrà utilizzato per la comunicazione tra i nodi worker nella configurazione distribuita. Il prefisso `grpc://` indica che il server utilizza il protocollo gRPC per la comunicazione. La seconda riga indica che il nodo worker0 (replica 0, task 0) si è connesso al servizio di coordinamento.

Questo servizio è responsabile della gestione del processo di training distribuito, inclusa l'assegnazione dei task, la sincronizzazione e il controllo degli errori. Il valore `incarnation` è un identificatore univoco per questa connessione. Nella terza riga viene comunicato che l'agente di coordinamento, responsabile della comunicazione con il servizio di coordinamento, si è connesso correttamente al servizio. Ciò significa che il nodo `worker0` è ora pronto per partecipare al processo di training distribuito. Le righe successive sono i log degli altri nodi worker che si connettono al cluster per l'addestramento della rete. Quando tutti i nodi worker che sono indicati in `TF_CONFIG` sono pronti, comincia il training del modello.

Capitolo 8

Conclusioni

Riassumendo, il lavoro si è svolto in quattro macrofasi principali: il recupero di algoritmi Spark pre-esistenti, lo sviluppo di un modello in Tensorflow (con relativo studio delle modalità di distribuzione della computazione), l'esecuzione dei modelli su diverse architetture distribuite diverse, la valutazione delle performance.

L'attività di recupero degli algoritmi scritti in linguaggio Java ha consentito lo sviluppo di diversi modelli ML eseguibili autonomamente grazie all'utilizzo di Container Docker. Per questo motivo, tali modelli possono essere utilizzati come componenti per la costruzione di sistemi più complessi. Nel corso del lavoro è stata resa evidente la possibilità di allenare ed eseguire i modelli su infrastrutture distribuite differenti, senza necessità di intervenire sul codice degli algoritmi. Inoltre, è stato studiato un approccio semplice e scalabile per l'implementazione di nuovi modelli sviluppati in Tensorflow, eseguibili sulla stessa infrastruttura distribuita. Le attività svolte hanno dunque mostrato un metodo per costruire sistemi complessi per l'analisi di Big Data, offrendo la possibilità di riuso del codice di modelli già implementati, grazie al disaccoppiamento fra codice ed infrastruttura distribuita con Kubernetes. Gli aspetti comuni agli algoritmi Spark e Tensorflow, infatti, sono proprio la possibilità di esecuzione in Container Docker e la distribuzione del carico computazionale orchestrando tali Container con Kubernetes, sfruttando sia la modalità di distribuzione offerta da Spark, sia quella offerta dalla libreria Tensorflow. Questa tesi ha fornito, quindi, una panoramica sull'integrazione delle tecnologie Spark, Kubernetes e Tensorflow.

La ricerca svolta nella tesi offre diverse opportunità per futuri sviluppi. In primo luogo, si potrebbe esplorare l'implementazione di un sistema di scheduling dei flussi di lavoro come Apache Oozie o Apache Airflow per creare pipeline di task, consentendo la creazione di un'infrastruttura più adatta a procedure di analisi dati complesse e strutturate. In particolare, è da notare che lo

scheduler Apache Airflow offre una facile integrazione anche con i modelli Tensorflow. In secondo luogo, l'approccio proposto per l'implementazione di nuovi algoritmi utilizzando Python e Tensorflow si è dimostrato altamente scalabile, in quanto offre la possibilità di distribuire la computazione su infrastrutture esistenti, con minime modifiche al codice. Si apre dunque la strada allo sviluppo di ulteriori modelli per l'analisi dati, o all'ottimizzazione di algoritmi già esistenti e ben consolidati nella letteratura.

Appendice A

POM generale

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="
  http://www.w3.org/2001/XMLSchema-instance"
2     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  https://maven.apache.org/xsd/maven-4.0.0.xsd">
3   <modelVersion>4.0.0</modelVersion>
4
5   <groupId>it.unimi.evotion.tasks</groupId>
6   <artifactId>kmeans-task</artifactId>
7   <version>1.0</version>
8   <packaging>pom</packaging>
9
10  <modules>
11    <module>kmeans</module>
12    <module>kmeans_model</module>
13    <module>kmeans_predict</module>
14  </modules>
15 </project>
```

Appendice B

POM di un singolo modulo

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http
      ://maven.apache.org/xsd/maven-4.0.0.xsd">
5     <modelVersion>4.0.0</modelVersion>
6
7     <parent>
8         <groupId>it.unimi.evotion.tasks</groupId>
9         <artifactId>kmeans-task</artifactId>
10        <version>1.0</version>
11    </parent>
12
13    <artifactId>kmeans</artifactId>
14    <name>KMeans</name>
15    <version>1.0</version>
16
17    <properties>
18        <maven.compiler.source>1.8</maven.compiler.source>
19        <maven.compiler.target>1.8</maven.compiler.target>
20        <project.build.sourceEncoding>UTF-8</project.build.
      sourceEncoding>
21
22        <spark.version>3.5.0</spark.version>
23        <scala.version>2.12</scala.version>
24    </properties>
25
26    <build>
27        <plugins>
28            <plugin>
29                <artifactId>maven-assembly-plugin</artifactId>
30                <configuration>
31                    <archive>
```

```

32         <manifest>
33             <addClasspath>true</addClasspath>
34             <mainClass>it.unimi.evotion.tasks.kmeans.
                Kmeans</mainClass>
35         </manifest>
36     </archive>
37     <descriptorRefs>
38         <descriptorRef>jar-with-dependencies</
            descriptorRef>
39     </descriptorRefs>
40 </configuration>
41 <executions>
42     <execution>
43         <id>make-assembly</id>
44         <phase>package</phase>
45         <goals>
46             <goal>single</goal>
47         </goals>
48     </execution>
49 </executions>
50 </plugin>
51 <plugin>
52     <groupId>org.apache.logging.log4j</groupId>
53     <artifactId>log4j-transform-maven-plugin</artifactId>
54     <version>0.1.0</version>
55     <executions>
56         <execution>
57             <goals>
58                 <goal>process-classes</goal>
59             </goals>
60         </execution>
61     </executions>
62 </plugin>
63 </plugins>
64 </build>
65
66 <dependencies>
67     <dependency>
68         <groupId>org.apache.spark</groupId>
69         <artifactId>spark-core_${scala.version}</artifactId>
70         <version>${spark.version}</version>
71         <scope>provided</scope>
72     </dependency>
73     <dependency>
74         <groupId>org.apache.spark</groupId>
75         <artifactId>spark-sql_${scala.version}</artifactId>
76         <version>${spark.version}</version>
77         <scope>provided</scope>

```

```

78     </dependency>
79     <dependency>
80         <groupId>org.apache.spark</groupId>
81         <artifactId>spark-mllib_${scala.version}</artifactId>
82         <version>${spark.version}</version>
83         <scope>provided</scope>
84     </dependency>
85     <dependency>
86         <groupId>org.apache.spark</groupId>
87         <artifactId>spark-streaming_${scala.version}</
            artifactId>
88         <version>${spark.version}</version>
89         <scope>provided</scope>
90     </dependency>
91     <dependency>
92         <groupId>org.apache.logging.log4j</groupId>
93         <artifactId>log4j-api</artifactId>
94         <version>2.22.1</version>
95     </dependency>
96     <dependency>
97         <groupId>org.apache.logging.log4j</groupId>
98         <artifactId>log4j-core</artifactId>
99         <version>2.22.1</version>
100    </dependency>
101    <dependency>
102        <groupId>org.apache.spark.ml.feature</groupId>
103        <artifactId>VectorDisassembler</artifactId>
104        <version>1.0</version>
105    </dependency>
106    </dependencies>
107 </project>

```

Appendice C

IAM policies

Trust policy per la creazione del ruolo per l'esecuzione dei task, si consente al solo utente *admin* di assumere il ruolo:

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Principal": { "AWS": "arn:aws:iam::[ID]:user/admin" },
7       "Action": "sts:AssumeRole"
8     }
9   ]
10 }
```

Nota: rispetto al file originale, l'ID del cluster è stato sostituito da [ID].

Policy che garantisce l'accesso al bucket Amazon Simple Storage Service (S3) di logging (situato all'Amazon Resource Name indicato) e a CloudWatch, associata al ruolo sopracitato:

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "s3:PutObject",
8         "s3:GetObject",
9         "s3:ListBucket"
10      ],
11       "Resource": [
12         "arn:aws:s3:::logs-tasks-stentella",
13         "arn:aws:s3:::logs-tasks-stentella/*"
14      ]
15     },
```

```

16     {
17         "Effect": "Allow",
18         "Action": [
19             "logs:CreateLogStream",
20             "logs:DescribeLogGroups",
21             "logs:DescribeLogStreams"
22         ],
23         "Resource": [
24             "arn:aws:logs:*:*:*"
25         ]
26     },
27     {
28         "Effect": "Allow",
29         "Action": [
30             "logs:PutLogEvents"
31         ],
32         "Resource": [
33             "arn:aws:logs:*:*:log-group:task-group:log-stream:
              task/*"
34         ]
35     }
36 ]
37 }

```

Policy per l'amministrazione dei cluster virtuali e il lancio dei Job:

```

1  {
2      "Version": "2012-10-17",
3      "Statement": [
4          {
5              "Effect": "Allow",
6              "Action": [
7                  "iam:CreateServiceLinkedRole"
8              ],
9              "Resource": "*",
10             "Condition": {
11                 "StringLike": {
12                     "iam:AWSServiceName": "emr-containers.amazonaws.
                      com"
13                 }
14             }
15         },
16         {
17             "Effect": "Allow",
18             "Action": [
19                 "emr-containers:CreateVirtualCluster",
20                 "emr-containers:ListVirtualClusters",
21                 "emr-containers:DescribeVirtualCluster",
22                 "emr-containers>DeleteVirtualCluster"
23             ],

```

```
24         "Resource": "*"
25     },
26     {
27         "Effect": "Allow",
28         "Action": [
29             "emr-containers:StartJobRun",
30             "emr-containers:ListJobRuns",
31             "emr-containers:DescribeJobRun",
32             "emr-containers:CancelJobRun"
33         ],
34         "Resource": "*"
35     }
36 ]
37 }
```

Appendice D

Configurazione di EKS StartJobRun

```
1 {
2   "name": "kmeans",
3   "virtualClusterId": "[VIRTUAL-ID]",
4   "executionRoleArn": "arn:aws:iam::[ID]:role/EMRContainers-
      JobExecutionRole",
5   "releaseLabel": "emr-7.0.0-latest",
6   "jobDriver": {
7     "sparkSubmitJobDriver": {
8       "entryPoint": "arn:aws:s3:::test-tasks-stentella/kmeans
      -1.0-jar-with-dependencies.jar",
9       "entryPointArguments": ["s3://test-tasks-stentella/
      dataset.csv", "5",
10      "s3://test-tasks-stentella/result"],
11      "sparkSubmitParameters": "\
12      --master k8s://https://E5F648AFE82BAF20017B44F112918FBE
      .gr7.us-east-1 \
13      .eks.amazonaws.com \
14      --class it.unimi.evotion.tasks.kmeans.Kmeans \
15      --conf spark.kubernetes.container.image= \
16      755674844232.dkr.ecr.us-east-1.amazonaws.com/spark/emr
      -7.0.0:latest \
17      --conf spark.executor.instances=5 \
18      --deploy-mode cluster"
19     }
20   },
21   "configurationOverrides": {
22     "applicationConfiguration": [
23       {
24         "classification": "spark-defaults",
25         "properties": {
26           "spark.driver.memory": "2G"
```

```

27         }
28     }
29 ],
30     "monitoringConfiguration": {
31         "persistentAppUI": "ENABLED",
32         "cloudWatchMonitoringConfiguration": {
33             "logGroupName": "task-group",
34             "logStreamNamePrefix": "task"
35         },
36         "s3MonitoringConfiguration": {
37             "logUri": "s3://logs-tasks-stentella"
38         }
39     }
40 }
41 }

```

Nota: rispetto al file originale, l'ID del cluster virtuale è stato sostituito da [VIRTUAL-ID], quello del cluster è stato sostituito da [ID].

Appendice E

Estratto dai log di un'esecuzione su cluster EKS

```
1 [...]
2 24/03/02 08:11:55 INFO ExecutorPodsAllocator: Going to request 4
   executors from Kubernetes for ResourceProfile Id: 0, target:
   5, known: 0, sharedSlotFromPendingPods: 2147483647.
3 [...]
4 24/03/02 08:12:02 INFO
   KubernetesClusterSchedulerBackend$KubernetesDriverEndpoint:
   Registered executor NettyRpcEndpointRef(spark-client://
   Executor) (192.168.3.216:50974) with ID 1, ResourceProfileId
   0
5 24/03/02 08:12:02 INFO BlockManagerMasterEndpoint: Registering
   block manager 192.168.3.216:46787 with 408.9 MiB RAM,
   BlockManagerId(1, 192.168.3.216, 46787, None)
6 24/03/02 08:12:02 INFO
   KubernetesClusterSchedulerBackend$KubernetesDriverEndpoint:
   Registered executor NettyRpcEndpointRef(spark-client://
   Executor) (192.168.58.9:37568) with ID 2, ResourceProfileId 0
7 24/03/02 08:12:02 INFO
   KubernetesClusterSchedulerBackend$KubernetesDriverEndpoint:
   Registered executor NettyRpcEndpointRef(spark-client://
   Executor) (192.168.47.240:41520) with ID 4, ResourceProfileId
   0
8 24/03/02 08:12:03 INFO
   KubernetesClusterSchedulerBackend$KubernetesDriverEndpoint:
   Registered executor NettyRpcEndpointRef(spark-client://
   Executor) (192.168.27.77:37000) with ID 3, ResourceProfileId
   0
9 [...]
10 24/03/02 08:12:05 INFO TaskSchedulerImpl: Adding task set 0.0
    with 20 tasks resource profile 0
11 24/03/02 08:12:05 INFO TaskSetManager: Starting task 0.0 in
```

```

    stage 0.0 (TID 0) (192.168.58.9, executor 2, partition 0,
    PROCESS_LOCAL, 1007791 bytes)
12 24/03/02 08:12:05 INFO TaskSetManager: Starting task 1.0 in
    stage 0.0 (TID 1) (192.168.3.216, executor 1, partition 1,
    PROCESS_LOCAL, 1007796 bytes)
13 24/03/02 08:12:05 INFO TaskSetManager: Starting task 2.0 in
    stage 0.0 (TID 2) (192.168.27.77, executor 3, partition 2,
    PROCESS_LOCAL, 1007796 bytes)
14 24/03/02 08:12:05 INFO TaskSetManager: Starting task 3.0 in
    stage 0.0 (TID 3) (192.168.47.240, executor 4, partition 3,
    PROCESS_LOCAL, 1007796 bytes)
15 [...]
16 24/03/02 08:12:08 INFO SparkContext: SparkContext is stopping
    with exitCode 0.
17 24/03/02 08:12:08 INFO SparkUI: Stopped Spark web UI at http://
    spark-0000000033fu6vq729gn-17753c8dfe38181f-driver-svc.task-
    namespace.svc:4040
18 24/03/02 08:12:08 INFO KubernetesClusterSchedulerBackend:
    Shutting down all executors
19 [...]

```


Glossario

algoritmi all-reduce algoritmi utilizzati nell'ambito della computazione parallela, che aggregano gli *array* (o i tensori) utilizzati da processi indipendenti in un'unica struttura dati. 34

Amazon Simple Storage Service (S3) File system distribuito offerto da AWS. III, 9, 15, 30, 32, 52

Apache Airflow Scheduler sviluppato da Airbnb e reso *open source*. III, 46, 47

Big Data Insiemi di dati ad alta dimensionalità, spesso non strutturati, prodotti in maniera continuativa da un servizio o da dispositivi IoT. III, 1, 2, 5, 6, 46

Bigtable Sistema di archiviazione distribuito per dati strutturati. III

cluster Insieme di risorse computazionali. III, IV, VI, 3, 6, 9, 11-15, 26-37, 39-43, 45, 61

Container Unità di virtualizzazione software che contiene un software unitamente alle sue dipendenze, senza virtualizzare un intero sistema operativo. 10-12, 14, 15, 26, 28-32, 36, 37, 39, 42, 46, 60, 61

data pipeline Sequenza di task che effettuano operazioni sui dati, in cui l'output di ogni task è usato come input dal successivo. 61

Docker Strumento per eseguire e amministrare i Container. III, 10, 11, 14, 15, 26, 31, 37, 39, 41, 42, 46

file system Struttura dati utilizzata per la gestione dei dati archiviati su un supporto elettronico. III, 60

Hadoop Yarn Yet Another Resource Negotiator, è la tecnologia di gestione delle risorse e pianificazione dell'esecuzione task nel framework di elaborazione distribuita open source Hadoop. III, 2, 9

Kubernetes Orchestratore per gestire un cluster composto da Container. III, IV, VI, 2, 3, 9, 11–15, 26–29, 31, 33, 36, 37, 39–44, 46

Luigi Scheduler sviluppato da Spotify e reso *open source*. III

MapReduce Modello di programmazione per processare in maniera semplice operazioni, svolte su grandi dataset, eseguite su cluster di grandi dimensioni. III

scheduler Strumento per la gestione di flussi di lavoro, nell’ambito specifico di questo elaborato, si intende per l’ingegnerizzazione di data pipeline. III, 60, 61

YAML linguaggio dalla sintassi minimale per la serializzazione dei dati, spesso utilizzato nei file di configurazione. 11, 13, 36, 37

Acronimi

ANOVA ANalysis Of VAriance. 20, 21

API Application Programming Interface. IV, 5-7, 9-13, 23, 26, 27, 33, 35, 39-41

ARN Amazon Resource Name. 32

AWS Amazon Web Services. 13, 15, 28, 30, 31, 41, 43, 60

CLI Command Line Interface. 12, 13, 28

CNN Convolutional Neural Network. 14, 35

CPU Central Processing Unit. 24, 26, 29, 34, 36, 37, 40

DAG Directed Acyclic Graph. 7, 8

DL Deep Learning. IV, 1, 5, 6, 9, 14, 33

DoS Denial of Service. 2

EC2 Elastic Compute Cloud. 29

EKS Elastic Kubernetes Service. 15, 28-31, 41, 43

EMR Elastic MapReduce. III, 28, 29, 32

EMRFS EMR File System. 32

GLM Generalized Linear Model. 21, 22

GLR Generalized Linear Regression. 22

GMM Gaussian Mixture Model. 18

GPU Graphics Processing Unit. 34, 36, 37, 39

HDFS Hadoop Distributed File System. 32

IA Intelligenza Artificiale. 4

IAM Identity and Access Management. VII, 9, 28, 30, 52

IoT Internet of Things. 2

ML Machine Learning. III, IV, 1, 2, 5, 9, 10, 13–15, 17, 22, 24, 42, 46

MLlib Machine Learning library. III, 1, 17

MUSA Multilayered Urban Sustainability Action. 2, 3, 13, 15, 40, 41

NCCL NVIDIA Collective Communications Library. 34, 35

POM Project Object Model. 24, 25

RDD Resilient Distributed Datasets. 7, 8

RPC Remote Procedure Call. 34, 35, 39

SLR Simple Linear Regression. 21, 22

SQL Structured Query Language. 40

TPU Tensor Processing Unit. 34, 35

UI User Interface. 12

URL Uniform Resource Locator. 12, 27, 41

VM Virtual Machines. 10

XML eXtensible Markup Language. 24

Bibliografia

- [1] Martin Abadi et al. “{TensorFlow}: A System for {Large-Scale} Machine Learning”. In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. 2016, pp. 265–283. isbn: 978-1-931971-33-1.
- [2] Hervé Abdi e Lynne J. Williams. “Principal Component Analysis”. In: *WIREs Computational Statistics* 2.4 (2010), pp. 433–459. issn: 1939-5108, 1939-0068. doi: 10.1002/wics.101.
- [3] Marco Anisetti et al. *Big Data Analytics Engine , Deliverable D5.5 to the EVOTION-727521 Project Funded by the European Union*. UNIMI, Italy, 2018.
- [4] Marco Anisetti et al. “MUSA: A Platform for Data-Intensive Services in Edge-Cloud Continuum”. In: *Advanced Information Networking and Applications*. A cura di Leonard Barolli. Vol. 203. Cham: Springer Nature Switzerland, 2024, pp. 327–337. isbn: 978-3-031-57930-1 978-3-031-57931-8. doi: 10.1007/978-3-031-57931-8_32.
- [5] *Apache Maven Assembly Plugin – Introduction*. 2024. url: <https://maven.apache.org/plugins/maven-assembly-plugin/>.
- [6] Brendan Burns et al. “Borg, Omega, and Kubernetes”. In: *ACM queue : tomorrow’s computing today* 14 (2016), pp. 70–93.
- [7] Fay Chang et al. “Bigtable: A Distributed Storage System for Structured Data”. In: *ACM Trans. Comput. Syst.* 26.2 (2008). issn: 0734-2071. doi: 10.1145/1365815.1365816.
- [8] Li Chen et al. “Fast Kernel k -Means Clustering Using Incomplete Cholesky Factorization”. In: *Applied Mathematics and Computation* 402 (2021), p. 126037. issn: 00963003. doi: 10.1016/j.amc.2021.126037.
- [9] Xiaohui Chen e Yun Yang. “Diffusion K-means Clustering on Manifolds: Provable Exact Recovery via Semidefinite Relaxations”. In: *Applied and Computational Harmonic Analysis* 52 (2021), pp. 303–347. issn: 10635203. doi: 10.1016/j.acha.2020.03.002.

- [10] D. Coomans e D.L. Massart. “Alternative K-Nearest Neighbour Rules in Supervised Pattern Recognition”. In: *Analytica Chimica Acta* 136 (1982), pp. 15–27. issn: 00032670. doi: 10.1016/S0003-2670(01)95359-0.
- [11] T. Cover e P. Hart. “Nearest Neighbor Pattern Classification”. In: *IEEE Transactions on Information Theory* 13.1 (1967), pp. 21–27. issn: 0018-9448, 1557-9654. doi: 10.1109/TIT.1967.1053964.
- [12] Jeffrey Dean e Sanjay Ghemawat. “MapReduce: Simplified Data Processing on Large Clusters”. In: *Communications of The Acm* 51.1 (2008), pp. 107–113. issn: 0001-0782. doi: 10.1145/1327452.1327492.
- [13] *Distributed Training with TensorFlow | TensorFlow Core*. url: https://www.tensorflow.org/guide/distributed%5C_training.
- [14] Annette J. Dobson e Adrian G. Barnett. *An Introduction to Generalized Linear Models*. Fourth edition. Chapman & Hall/CRC Texts in Statistical Science Series. Boca Raton: CRC Press, Taylor & Francis Group, 2018. isbn: 978-1-138-74151-5 978-1-138-74168-3.
- [15] *EMR File System (EMRFS) - Amazon EMR*. url: <https://docs.aws.amazon.com/emr/latest/ReleaseGuide/emr-fs.html>.
- [16] Jianqing Fan, Fang Han e Han Liu. “Challenges of Big Data Analysis”. In: *National Science Review* 1.2 (2014), pp. 293–314. issn: 2053-714X, 2095-5138. doi: 10.1093/nsr/nwt032.
- [17] Sanjay Ghemawat, Howard Gobioff e Shun-Tak Leung. “The Google File System”. In: *Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles*. Sosp ’03. New York, NY, USA: Association for Computing Machinery, 2003, pp. 29–43. isbn: 1-58113-757-5. doi: 10.1145/945445.945450.
- [18] David W. Hosmer, Stanley Lemeshow e Rodney X. Sturdivant. *Applied Logistic Regression*. Third edition. Wiley Series in Probability and Statistics 398. Hoboken, New Jersey: Wiley, 2013. isbn: 978-1-118-54835-6.
- [19] Abiodun M. Ikotun et al. “K-Means Clustering Algorithms: A Comprehensive Review, Variants Analysis, and Advances in the Era of Big Data”. In: *Information Sciences* 622 (2023), pp. 178–210. issn: 00200255. doi: 10.1016/j.ins.2022.11.139.
- [20] Jasmin Praful Bharadiya. “Machine Learning and AI in Business Intelligence: Trends and Opportunities”. In: *International Journal of Computer (IJC)* 48.1 (2023), pp. 123–134.
- [21] Yann LeCun, Corinna Cortes e Christopher J.C. Burges. *The MNIST Database of Handwritten Digits*. 1994.

- [22] *Log4j Transform*. 2023. url: <https://logging.apache.org/log4j/transform/latest/>.
- [23] Sergey Melnik et al. “Dremel: Interactive Analysis of Web-Scale Datasets”. In: *Proc. of the 36th Int’l Conf on Very Large Data Bases*. 2010, pp. 330–339.
- [24] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. doi: 10.48550/ARXIV.1912.01703.
- [25] Jean-Georges Perrin e Rob Thomas. *Spark in Action*. Second edition. Shelter Island, NY: Manning Publications Co, 2020. isbn: 978-1-61729-552-2.
- [26] N.A. Priyanka e Dharmender Kumar. “Decision Tree Classifier: A Detailed Survey”. In: *International Journal of Information and Decision Sciences* 12.3 (2020), p. 246. issn: 1756-7017, 1756-7025. doi: 10.1504/IJIDS.2020.108141.
- [27] Douglas Reynolds. “Gaussian Mixture Models”. In: *Encyclopedia of Biometrics*. A cura di Stan Z. Li e Anil K. Jain. Boston, MA: Springer US, 2015, pp. 827–832. isbn: 978-1-4899-7487-7 978-1-4899-7488-4. doi: 10.1007/978-1-4899-7488-4_196.
- [28] Salman Salloum et al. “Big Data Analytics on Apache Spark”. In: *International Journal of Data Science and Analytics* 1.3-4 (2016), pp. 145–164. issn: 2364-415X, 2364-4168. doi: 10.1007/s41060-016-0027-9.
- [29] Iqbal H. Sarker. “Machine Learning for Intelligent Data Analysis and Automation in Cybersecurity: Current and Future Prospects”. In: *Annals of Data Science* 10.6 (2023), pp. 1473–1498. issn: 2198-5804, 2198-5812. doi: 10.1007/s40745-022-00444-2.
- [30] Henry Scheffé. *The Analysis of Variance*. Wiley classics library ed. A Wiley Publication in Mathematical Statistics. New York: Wiley-Interscience Publication, 1999. isbn: 978-0-471-34505-3 978-0-471-75834-1.
- [31] George A. F. Seber e Alan J. Lee. *Linear Regression Analysis*. 2nd ed. Hoboken: John Wiley & Sons, 2012. isbn: 978-1-118-27442-2.
- [32] Ketan Rajshekhar Shahapure e Charles Nicholas. “Cluster Quality Analysis Using Silhouette Score”. In: *2020 IEEE 7th International Conference on Data Science and Advanced Analytics (DSAA)*. sydney, Australia: IEEE, 2020, pp. 747–748. isbn: 978-1-72818-206-3. doi: 10.1109/DSAA49011.2020.00096.
- [33] Xiaobo Shen et al. “Compressed K-Means for Large-Scale Clustering”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 31.1 (2017). issn: 2374-3468, 2159-5399. doi: 10.1609/aaai.v31i1.10852.

- [34] Sidney Siegel. “Nonparametric Statistics”. In: *The American Statistician* 11.3 (1957), pp. 13–19. issn: 0003-1305, 1537-2731. doi: 10.1080/00031305.1957.10501091.
- [35] Michael Steinbach, George Karypis e Vipin Kumar. “A Comparison of Document Clustering Techniques”. In: (2000).
- [36] Alaa Tharwat et al. “Linear Discriminant Analysis: A Detailed Tutorial”. In: *AI Communications* 30.2 (2017), pp. 169–190. issn: 18758452, 09217126. doi: 10.3233/AIC-170729.
- [37] Xue-Wen Chen e Xiaotong Lin. “Big Data Deep Learning: Challenges and Perspectives”. In: *IEEE Access* 2 (2014), pp. 514–525. issn: 2169-3536. doi: 10.1109/ACCESS.2014.2325029.
- [38] Petar Zečević e Marko Bonaći. *Spark in Action*. Shelter Island: Manning, 2017. isbn: 978-1-61729-260-6.